BTNR - Bayesian Tensor Networks for Regression

Anonymous authors
Paper under double-blind review

Abstract

Universal function approximation via basis expansions such as Fourier or polynomial bases is resource intensive. While the number of coefficients grows moderately in low dimensions, it can scale exponentially with the input size, leading to high complexity and potential overfitting. Tensor networks compress the coefficient space allowing for linear rather than exponential scaling. Existing tensor networks methods derive their learning equations for one specific network topology and few explore Bayesian learning methods. Probabilistic frameworks naturally derive confidence estimates for model predictions, allowing real world applications to account for uncertainties in the decision process. We propose Bayesian Tensor Networks for Regression (BTNR), a variational Bayesian inference framework whose analytical update equations are topology agnostic. A single derivation applies to a large class of tensor network structures, ~~independently to~~ independent of the feature map. BTNR also recovers standard ridge regularized alternating least squares (ALS) inference as a special case. Within the framework, automatic relevance determination (ARD) can learn the relevance of compression hyperparameters during training and prunes irrelevant structure, removing the need for a part of the costly ablation over hyperparameters that classical model estimation requires. The Bayesian formulation further yields predictive uncertainty directly from the inferred posterior, without recourse to a separate estimation procedure. We benchmark BTNR on multivariate polynomial regression across four tensor network topologies and contrast it against ridge regularized ALS as well as non tensor network based standard Bayesian regression baselines. BTNR attains point estimates comparable to hyperparameter tuned ALS while avoiding the hyperparameter search, and delivers well calibrated predictive uncertainty, reaching the lowest calibration error on several datasets with respect to the Bayesian baselines. Importantly, BTNR provides a universal foundation for exploring tensor network structures in a supervised learning context.

1 Introduction

Learning the relation that maps input data to their associated outputs is one of the most studied problems in machine learning. It is well known that any function can be approximated as a linear combination of a suitable functional base. The challenge arises from the often exponentially large space of coefficients in the selected functional base.

Tensor networks are structures that compress the multidimensional space of tensors into a more tractable subspace that grows linearly with the dimensions of the tensor instead of exponentially. A variety of methods to approximate coefficients for functional approximations have been explored, principally the structures being canonical polyadic decomposition (CPD), tensor train (TT) or matrix product states (MPS), and matrix product operators (MPOs), see (Batselier, 2023; Hendrikx et al., 2019; Govindarajan et al., 2022; Ayvaz and De Lathauwer, 2022; Stoudenmire and Schwab, 2016; Götte et al., 2021; Kilic and Batselier, 2025; Ciolli et al., 2026; Batselier et al., 2016).

Non probabilistic methods for tensor networks in general require parameter search over a high number of hyperparameters through cross-validation or extensive ablation studies. This can be especially expensive when paired with traditional algorithms for estimating the tensor network such as alternating least squares (ALS) or density matrix renormalization group (DMRG) (Holtz et al., 2012; Grasedyck et al., 2019), which

can be computationally demanding. Recently, Bayesian modeling has been explored in this context, in particular leveraging automatic relevance determination (ARD) to robustly learn model structure from suitable high rank representations (Kilic and Batselier, 2025; Mørup and Hansen, 2009; Hinrich et al., 2020), thereby reducing the need for hyperparameter search over structural properties of the tensor network, namely bond dimensions and regularization parameters. A key advantage of Bayesian modeling, is the inherited uncertainty estimation to evaluate the confidence of a prediction, which in many real world cases can be an essential deciding factor. Additionally, ~~non Bayesian~~ maximum likelihood based frameworks often require careful hyperparameters tuning, especially when regularization is introduced, usually necessary to obtain good generalization.

A common limitation, however, is that tensor network methods are commonly tailored to a specific model. Their learning equations are derived for one chosen topology, most often CPD or tensor train, and do not naturally transfer to other structures. The derivations in our work are instead obtained without any structural assumption, yielding update equations that apply unchanged to any tensor network topology and any feature map. Topology independence most distinguishes our framework from prior Bayesian tensor network inference, which are specific to certain decompositions (Kilic and Batselier, 2025; Konstantinidis et al., 2022; Hinrich et al., 2020). The result is a universal framework for Bayesian inference over the majority of tensor network structures, independent of both the topology and the basis.

With the rise of tensor networks computational frameworks supporting easy manipulation of arbitrary tensor network structures, generalized equations are often straightforward to implement. One such tool is the library quimb Gray (2018), which is used by this paper to implement most of the code use in the experiments, which is provided in Anonymous Repository (2026).

~~The result is a universal framework for Bayesian inference over the majority of tensor network structures, independent of both the topology and the basis.~~

~~The~~ developed framework is benchmarked on a set of experiments for multivariate polynomial regression to contrast the quality of the predictions against the standard alternating least squares (ALS) procedure and baselines (Ciolli et al., 2026). Additionally the quality of estimated uncertainties is contrasted against non tensor network Bayesian learning baselines.

Specifically, our contributions are as follows:

- We provide analytical Bayesian inference update equations, that work universally for any tensor network structure and basis, also naturally accommodating traditional ALS procedures with ridge regularization.
- Using this generic framework, we implement a series of existing tensor network based polynomial regression procedures based on the canonical polyadic decomposition, tensor train/MPS, and MPOs. To highlight the versatility of our framework we also consider the binary tensor tree (BTT)~~, which has~~ (Cheng et al., 2019) and tensor ring (TR) (Zhao et al., 2016), which have not previously been used in the context of multivariate polynomial regression.
- We improve efficiency by dynamically pruning irrelevant terms through automatic relevance determination, eliminating the need for costly ablation studies. We systematically compare the implemented Bayesian tensor network approaches for polynomial regression across a range of ten tabular problems, including direct comparisons with their maximum-likelihood (ALS) counterparts as well as non tensor network Bayesian baselines.
- We evaluate the predictive uncertainty produced by our framework against non tensor network Bayesian baselines, observing that the method is well calibrated and sharp, attaining the lowest expected calibration error on several datasets.

2 Related Works

Several works have explored tensor networks as a method to perform polynomial regression, where Ayvaz and De Lathauwer (2022) propose different types of CPD decomposition based structures, learning the

polynomial coefficients through efficient Gauss-Newton learning methods. Tensor train structures have further been explored in (Götte et al., 2021; Batselier et al., 2016). Furthermore, the application of Gauss-Newton and gradient methods to learn the coefficients for polynomial regression and classification can be achieved using stacked matrix product operators (MPOs) structures (Ciolli et al., 2026).

Bayesian approaches have been used to perform inference on tensor networks, mostly focusing on CPD or tensor train structures using Variational Bayesian inference and Gibbs sampling, Hinrich and Mørup (2019); Hinrich et al. (2020). The work of Guo and Draper (2026) explores initialization methods to ensure stability in tensor train models, using Bayesian inference with the Laplace method. In Konstantinidis et al. (2022), the posterior distribution weights are decomposed as a CPD and learned through stochastic gradient descent to minimize the target Kullback-Leibler divergence. The work of Kilic and Batselier (2025) derives a method to learn the posterior distribution of the parameters, through variational inference, with the parameters structured as CPD or tensor trains assuming mean field approximation and utilizing polynomial basis.

We presently extend the above methods to Bayesian inference for general tensor network structures, and we additionally study a symmetric polynomial base, where the modes of the tensor represents the features and the number of modes the degree of the polynomial as in Ciolli et al. (2026).

As far as our knowledge all existing methods present their work, and their equations for a specific tensor network structure, while our methodology is naturally applicable to any topology.

3 Background

In this section, we establish the notation utilized throughout this work. A scalar is denoted by a lower-case letter, such as a. A tensor $\mathbf{A} \in \mathbb{R}^{d_1 \times \cdots \times d_n}$ is a multi-linear array represented by a bold capital letter, where n denotes the order of the tensor and $d_1, \ldots, d_n$ specify its dimensions. For $n = 1$ and $n = 2$, the tensor represents a vector and a matrix, respectively.

To formulate equations applicable to arbitrary tensor network topologies, we define tensor contraction implicitly through shared modes, avoiding cumbersome explicit indexing.

A tensor network can be defined as a graph where nodes represent tensors and edges denote the dimensions of the tensors, where a free edge represents an uncontracted dimension, while shared edges represent tensor contraction between the two nodes across the shared dimension. Therefore a mode belongs to a tensor in the same way an edge connecting nodes in a graph belongs to a node. Thus, we refer to the modes of a tensor as its edges. Formally, for a tensor $\mathbf{A} \in V_1 \otimes V_2 \otimes \cdots \otimes V_n$, where each vector space $V_j \cong \mathbb{R}^{d_j}$, the edge associated with the j-th mode is indexing in the space V_j. Letting $\{\mathbf{e}_k^j\}_{k=1}^{d_j}$ denote the standard basis vectors for V_j, we loosely write $\mathbf{e}^j \in \mathbf{A}$ to indicate that these vectors form the basis for the j-th mode of $\mathbf{A}$.

The complete set of edges for a tensor $\mathbf{A}$ is defined as $\bar{E}(\mathbf{A}) = \{\boldsymbol{e}^j \mid \boldsymbol{e}^j \in \mathbf{A}\}$. For any two tensors $\mathbf{A}$ and $\mathbf{B}$, their shared edges are given by the intersection $C_{\mathbf{AB}} = \bar{E}(\mathbf{A}) \cap \bar{E}(\mathbf{B})$, while their respective unshared edges are $O(\mathbf{A}) = \bar{E}(\mathbf{A}) \setminus C_{\mathbf{AB}}$ and $O(\mathbf{B}) = \bar{E}(\mathbf{B}) \setminus C_{\mathbf{AB}}$. The contraction between $\mathbf{A}$ and $\mathbf{B}$, denoted simply as $\mathbf{AB}$, is defined as the product and summation over all shared dimensions, yielding a tensor belonging to the space of unshared edges $O(\mathbf{A}) \cup O(\mathbf{B})$. When all edges are shared the contraction results in a scalar and we denote it as $\langle \mathbf{AB} \rangle$. The explicit contraction is given by:

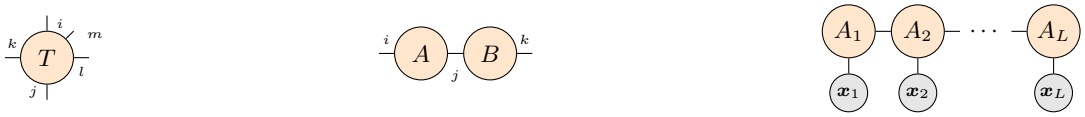
$$\mathbf{AB}_{O(\mathbf{A})\cup O(\mathbf{B})} = \sum_{\boldsymbol{e}_k^1 \in \boldsymbol{e}^1} \cdots \sum_{\boldsymbol{e}_l^n \in \boldsymbol{e}^n} \mathbf{A}_{O(\mathbf{A}),\boldsymbol{e}_k^1,\ldots,\boldsymbol{e}_l^n} \mathbf{B}_{O(\mathbf{B}),\boldsymbol{e}_k^1,\ldots,\boldsymbol{e}_l^n}, \tag{1}$$

where $\{\boldsymbol{e}^1, \ldots, \boldsymbol{e}^n\} \in C_{\mathbf{AB}}$ are the shared edges.

~~The implicit~~ The implicit contraction notation $\mathbf{AB}$ assumes summation over all shared dimensions, making redundant the explicit Equation 1.

Under this formulation, tensor contraction behaves identically to scalar multiplication, maintaining commutativity and associativity.

During this work tensor networks will be exclusively defined by the nodes and the relation between nodes and edges. Specifically, a tensor network $\mathbf{A}$ is composed by a set of nodes $\{\mathbf{A}^1, \ldots, \mathbf{A}^n\}$, a set of mode bases



(a) 5-d tensor T_{ijklm} (b) Matrix product: $T_{ik} = \sum_j A_{ij}B_{jk}$ (c) Contraction of two tensor networks $\mathbf{AX}$

Figure 1: Graphical representation of (a) the element T_{ijklm} of the tensor $\mathbf{T}$, (b) the matrix product, and (c) a tensor network $\mathbf{A}$ contracting a tensor $\mathbf{X}$.

$\{\boldsymbol{e}^1, \ldots, \boldsymbol{e}^m\}$ and a corresponding set of mode inclusions $\{\bar{E}(\mathbf{A}^1), \ldots, \bar{E}(\mathbf{A}^n)\}$. Equivalently to $\bar{E}(\cdot)$, we can define $\bar{N}(\boldsymbol{e}^j) = \{\mathbf{A}^i \mid \boldsymbol{e}^j \in \mathbf{A}^i\}$ as the set of nodes related to a particular edge. We use $\bar{E}_{\mathbf{A}}$ to indicate the set of all edges defining the tensor network $\mathbf{A}$, and $\bar{N}_{\mathbf{A}}$ to indicate the set of all nodes. Important is to distinguinsh $\bar{E}_{\mathbf{A}}$ and $\bar{E}(\mathbf{A})$, where the latter indicates the edges of $\mathbf{A}$ as a tensor, i.e. the edges belonging to the tensor resulting from the contraction of the tensor network.

The result of the contraction of the tensor network excluding node i, is indicated as $\hat{\mathbf{A}}^i$ and will be referred to as the environment of node $\mathbf{A}^i$. The edges of the resulting contraction of the environment of $\mathbf{A}^i$ are given by the union of the edges of the node and the edges of the tensor network, precisely:

$$\bar{E}(\hat{\mathbf{A}}^i) = \{\boldsymbol{e}^j \mid \boldsymbol{e}^j \in \bar{E}(\mathbf{A}^i) \cup \bar{E}(\mathbf{A})\}$$

Most operations require only the label of a mode. We define $E(i) = \{j \mid \boldsymbol{e}^j \in \mathbf{A}^i\}$ as the set of edge labels connected to node $\mathbf{A}^i$, and $N(j) = \{i \mid \boldsymbol{e}_j \in \mathbf{A}^i\}$ as the set of node indices contracting through edge j.

Additionally we remark that addition and subtraction of two tensors, can happen exclusively between tensors belonging to the same space, meaning with the same edges, since they are elementwise operations.

This notation abstracts away the exact shape and axis ordering of the tensors, requiring the definition of shared and unshared dimensions only for practical applications.

Conventional literature often focuses on specific structures and relies on reshaping tensors into matrices to utilize Kronecker, Khatri-Rao, or Hadamard products via standard linear algebra libraries. We argue that there are many efficient coding frameworks, which can perform linear algebra operations on tensor networks, simply by stating the modes to contract, allowing for easy implementation of structure agnostic equations, in the likes of the ones presented in this paper.

Figure 1 provides simple examples of tensor network diagrams: (a) a five-mode tensor, (b) standard matrix multiplication (a contraction along one mode), and (c) the contraction of a tensor network $\mathbf{A}$ with a tensor derived by the tensor product $\mathbf{X} = \boldsymbol{x}_1 \otimes \ldots \otimes \boldsymbol{x}_N$.

4 Methods

We consider the generic regression problem, where the objective is to learn the tensor of coefficients $\mathbf{A}$, given a dataset of S samples and d features, such that for sample s, $y_s \in \mathbb{R}$ represents the regression target:

$$y_s = \langle \Phi(\boldsymbol{x}_s)\mathbf{A} \rangle + \epsilon_s, \tag{2}$$

where $\boldsymbol{x}_s \in \mathbb{R}^d$ is the feature vector for sample s, ϵ_s are i.i.d. random variables modeling noise in the predictions and $\Phi(\cdot)$ is a map of the inputs to an arbitrary embedding, resulting in a tensor.

In this work for the experimentations and in the explanatory images, we will consider a polynomial type basis defined in the following way: Given a constant polynomial degree L and polynomial embedding feature map $\Phi_L : \mathbb{R}^d \to \mathbb{R}^{\times_{l=1}^{L}(d+1)}$ defined as

$$\Phi_L(\boldsymbol{x}_s) = \mathbf{X}_s = \tilde{\boldsymbol{x}}_s \otimes \cdots \otimes \tilde{\boldsymbol{x}}_s \in \mathbb{R}^{\times_{l=1}^{L}(d+1)}, \tag{3}$$

where $\tilde{\boldsymbol{x}}_s = [1, \boldsymbol{x}_s]$ is obtained by concatenating the scalar one to $\boldsymbol{x}_s$. The bias term is appended to the features so that the map $\Phi_L(\cdot)$ outputs a tensor $\mathbf{X}_s$ that contains all interactions between features of the

inputs up to degree L as considered by Ayvaz and De Lathauwer (2022). We drop the L label in $\mathbf{X}_s$, since it will be assumed constant through the derivation.

The regression with this basis is effectively finding the coefficients of the multivariate polynomial of degree L that best approximates the functional relation between inputs and outputs.

Importantly, this work proposes a methodology that is not dependent on the feature map. All the following derivations consider exclusively the final tensor $\mathbf{X}_s$ and do not exploit any underlying assumption of the shape and values of $\mathbf{X}_s$. Thus, the polynomial feature map can be trivially changed.

As often is the case with feature maps, the image of the feature map is often high-dimensional and practically untractable. To avoid the possible exponential size of the coefficient space, in this paper we model the coefficient tensor $\mathbf{A}$ as a tensor network.

4.1 Bayesian inference for tensor network regression

With the problem defined, we now introduce a Bayesian learning approach for regression, applicable to any suitable tensor network representation of the coefficients. Our framework models tensor networks where each node is unique, meaning nodes cannot be imposed to have identical weights. Imposing node equality in tensor networks is rarely required and often an unconstrained tensor network is generally sufficient. The most common and widely used tensor network models are covered by this work, including canonic polyadic decomposition (CPD), matrix product states/tensor trains (MPS/TT) and tensor trees.

Bayesian methods require the definition of a prior and posterior distribution of the weight tensor $\mathbf{A}$, $p(\mathbf{A})$ and $q(\mathbf{A})$ respectively. A natural assumption for the tensor network decomposition of $\mathbf{A}$ is to consider the probability of each component of the tensor network graph factorizible. As a consequence, each node and edge is associated to a parametrized probability distribution. The resulting distribution can be seen as a direct result of the mean field approximation assumption (Blei et al., 2017). The goal of Bayesian learning is to infer the parameters of the posterior distribution of the weights. We will use the technique defined as moment matching, where the prior and posterior distributions are conjugate, belonging to the same probabilistic family (Šmídl and Quinn, 2007).

Let us first define the probabilistic parametrization of the edges forming the tensor network $\mathbf{A}$. We associate to each element of the base vectors of an edge, $\boldsymbol{e}_k^j$ a parameter λ_{jk} as well as an edge $\boldsymbol{e}^j$ to a vector of parameters $\boldsymbol{\lambda}_j = \{\lambda_{j1}, \dots, \lambda_{j\,\dim(\boldsymbol{e}^j)}\}$.

Under the mean field assumption we factorize the probability of the edge basis $\boldsymbol{e}^j = \{\boldsymbol{e}_k^j\}_{k=1}^{d_j}$ and assume that the parameters λ_{jk} are distributed according to the ~~gamma distribution~~ Gamma distribution $Ga(x)$, allowing us to write the prior p and posterior q distribution as

$$p(\boldsymbol{\lambda}_j) = \prod_{k=1}^{\dim(\boldsymbol{e}^j)} p(\lambda_{jk}), \qquad p(\lambda_{jk}) = Ga(\lambda_{jk} \mid \alpha_{jk}^p, \beta_{jk}^p), \qquad q(\lambda_{jk}) = Ga(\lambda_{jk} \mid \alpha_{jk}^q, \beta_{jk}^q) \tag{4}$$

The parameters can also be vectorized, following the same definition as $\boldsymbol{\lambda}_j$ as $\boldsymbol{\alpha}_j^\chi = \{\alpha_{j1}^\chi, \dots, \alpha_{j\,\dim(\boldsymbol{e}^j)}^\chi\}$ and $\boldsymbol{\beta}_j^\chi = \{\beta_{j1}^\chi, \dots, \beta_{j\,\dim(\boldsymbol{e}^j)}^\chi\}$, where the superscript $\chi \in \{p, q\}$ indicates if they are weights associated to the prior or posterior distribution, respectively.

Let us now formalize the probability distribution associated to the nodes of the tensor network. Under mean field assumption each node $\mathbf{A}^i$ has its own prior and posterior distribution. We parametrize the probability of the nodes as multivariate Gaussian distributions. Note that we can retain the shapes of the tensor when used as arguments and parameters of multivariate Gaussian distributions, due to the bijectivity of the vectorization map that trivially flattens the tensor into a vector.

To retain meaning and structure the covariance must be a tensor belonging to the tensor product of the space of the variable with itself, in a way that the matricization follows the same reshaping rule as the vectorization. As example consider the given tensor $\mathbf{B} \in V = V_1 \otimes \dots \otimes V_n$ with mean $\boldsymbol{\mu}_\mathbf{B} \in V$ and variance $\boldsymbol{\Sigma}_\mathbf{B} \in V \otimes V = V_1 \otimes \dots \otimes V_n \otimes V_1 \otimes \dots \otimes V_n$. This specification allows us to simply maintain the tensor shape in the multivariate Gaussian probability distribution formula.

Regarding the prior we assume that the nodes have mean zero and diagonal covariance. The covariance will be parametrized with λ_{jk}, and defined as the diagonalization of the tensor parametrizing the node as the edges composing it. Explicitly let us consider node $\mathbf{A}^i \in V$, then we can define the tensor

$$\mathbf{\Lambda}^{E(i)} = \bigotimes_{j \in E(i)} \boldsymbol{\lambda}_j \in V.$$

It can be noted that the diagonalization of $\tilde{\mathbf{\Lambda}} = diag(\mathbf{\Lambda}) \in V \otimes V$.

We can now define the prior and posterior of node $\mathbf{A}^i$ as

$$p(\mathbf{A}^i) = \mathcal{N}(\mathbf{0}, \tilde{\mathbf{\Lambda}}^{E(i)^{-1}}), \quad q(\mathbf{A}^i) = \mathcal{N}(\boldsymbol{\mu}^i, \boldsymbol{\Sigma}^i), \tag{5}$$

where the usual relation of mean and covariance is present

$$\boldsymbol{\mu}^i = \mathbb{E}_{q(\mathbf{A}^i)}[\mathbf{A}^i], \quad \boldsymbol{\Sigma}^i = \mathbb{E}_{q(\mathbf{A}^i)}[\mathbf{A}^i \otimes \mathbf{A}^i] - \boldsymbol{\mu}^i \otimes \boldsymbol{\mu}^i. \tag{6}$$

The last step required is to parametrize the likelihood and noise. The likelihood is directly derived from the model definition in Equation 2 representing a point in the image of a generic feature map given by $\mathbf{X}_s$ as

$$p(y_s \mid \langle \mathbf{A}\mathbf{X}_s \rangle, \tau) = \mathcal{N}(y_s \mid \langle \mathbf{A}\mathbf{X}_s \rangle, \tau^{-1}). \tag{7}$$

We choose a gaussian distribution parametrization, since it is the probabilistic equivalent of the squared norm error for non Bayesian regression.

The parameter τ is the noise precision (i.e., inverse variance) and controls the inferred noise and we assume it is distributed as a gamma distribution:

$$p(\tau) = Ga(\tau \mid \alpha_\tau^p, \beta_\tau^p), \quad q(\tau) = Ga(\tau \mid \alpha_\tau^q, \beta_\tau^q). \tag{8}$$

The set of learnable parameters Θ is composed by the parameters of all posterior distributions of the tensor network defined by:

$$\Theta = \{\boldsymbol{\alpha}_1^q, \ldots, \boldsymbol{\alpha}_m^q, \boldsymbol{\beta}_1^q, \ldots, \boldsymbol{\beta}_m^q, \boldsymbol{\mu}^1, \ldots, \boldsymbol{\mu}^n, \boldsymbol{\Sigma}^1, \ldots, \boldsymbol{\Sigma}^n, \alpha_\tau^q, \beta_\tau^q\}.$$

We can explicitly define the joint distribution (unnormalized posterior distribution) and the variational posterior distributions as

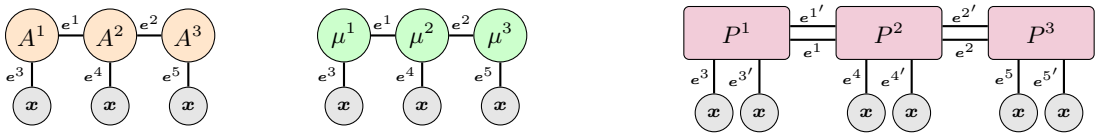
$$\begin{aligned} p(\boldsymbol{y}, \Theta) &= p(\tau) \prod_{s=1}^{S} p(y_s \mid \langle \mathbf{A}\mathbf{X}_s \rangle, \tau) \prod_{i \in N_\mathbf{A}} p(\mathbf{A}^i) \prod_{j \in E_\mathbf{A}} p(\boldsymbol{\lambda}_j), \\ q(\boldsymbol{y}, \Theta) &= q(\tau) \prod_{i \in N_\mathbf{A}} q(\mathbf{A}^i) \prod_{j \in E_\mathbf{A}} q(\boldsymbol{\lambda}_j), \end{aligned} \tag{9}$$

respectively.

The variational Bayesian inference framework then estimate parameters belonging to the distributions $q(\boldsymbol{\Theta})$ by maximizing the evidence lower-bound (ELBO) (Blei et al., 2017). The maximal ELBO is obtained by the optimal parameters solving the following equation,

$$\log p_i(\theta_i) \propto \mathbb{E}_{q(\Theta/\theta_i)}[\log p(\boldsymbol{y}, \Theta)], \tag{10}$$

up to a constant with respect to θ_i. This allows each independent subset of parameters θ_i to be updated conditioned on the value of all other parameters and iterating through the subsets multiple times is guaranteed to converge to a local optima.



(a) $\mathbf{A}$ network (b) $\boldsymbol{\mu}$ network (c) $\mathbf{P}$ network

Figure 2: Diagrammatic representation of how the topology of the $\boldsymbol{\mu}$ network and the ~~$\boldsymbol{\Sigma}$~~ $\mathbf{P}$ network is constructed given the $\mathbf{A}$ network defining the topology of the tensor network used for compressing the regression weight tensor.

4.2 Estimating the parameters of the variational distributions

While the tensor structure of $\mathbf{A}$ and the embedding $\Phi(\cdot)$ can be arbitrarily chosen, the choice of variational mean field approximation and prior distributions from the exponential family is what makes it possible to analytically find the updates for the parameters using moment matching.

Given a tensor network defined by the collection of nodes $\mathbf{A}^i$, edges $\boldsymbol{e}^j$, and how they connect, we define the mean and ~~covariance~~ the second moment tensor networks, $\boldsymbol{\mu}$ and ~~$\boldsymbol{\Sigma}$~~ $\mathbf{P}$ according to how the nodes are defined in Equation 6. Importantly, due to the definition of the nodes, the topology of $\boldsymbol{\mu}$ is inherited from $\mathbf{A}$, while the topology of ~~$\boldsymbol{\Sigma}$~~ $\mathbf{P}$ has the same structure as $\mathbf{A}$, but the basis of the edges are doubled. This means that the ~~$\boldsymbol{\Sigma}$~~ $\mathbf{P}$ tensor network, include in its edges all the edges of the $\boldsymbol{\mu}$ network. Consequently, $\boldsymbol{\mu}$ can contract with the input tensor $\mathbf{X}_s$, while ~~$\boldsymbol{\Sigma}$~~ $\mathbf{P}$ is contracted with $\mathbf{X}_s \otimes \mathbf{X}_s$. In Figure 2, we provide a graphical representation of the topology of the $\boldsymbol{\mu}$ and ~~$\boldsymbol{\Sigma}$~~ $\mathbf{P}$ networks derived from an MPS structured $\mathbf{A}$ network, and the specific polynomial map in Equation 3.

In the following we define $\mathbf{X}_s^{\mu} = \mathbf{X}_s$ and ~~$\mathbf{X}_s^{\Sigma} = \mathbf{X}_s \otimes \mathbf{X}_s$. In addition, let $\mathbf{P}$ be~~ $\mathbf{X}_s^{P} = \mathbf{X}_s \otimes \mathbf{X}_s$. Formally, the expected second moment network ~~, where each node is :~~ $\mathbf{P}$ is defined through its nodes as:

$$\mathbf{P}^i = \mathbb{E}_{q(\mathbf{A}^i)}[\mathbf{A}^i \otimes \mathbf{A}^i] = \boldsymbol{\Sigma}^i + \boldsymbol{\mu}^i \otimes \boldsymbol{\mu}^i = \mathbb{E}_{q(\mathbf{A}^i)}[\mathbf{A}^i \otimes \mathbf{A}^i]. \tag{11}$$

~~The input dimensions of $\boldsymbol{\Sigma}$ and $\mathbf{P}$ are identical by construction and in~~ In the code implementation it is only ~~necessary~~ needed to store in memory $\boldsymbol{\mu}^i$ and ~~either $\boldsymbol{\Sigma}^i$ or~~ $\mathbf{P}^i$ as the ~~one can be derived from the other using~~$\boldsymbol{\mu}^i$ node covariance can be directly derived using Equation 11.

We define the expectation of the tensor of the edges parameters as

$$\boldsymbol{\Gamma}^{E(i)} = \mathbb{E}[\boldsymbol{\Lambda}^{E(i)}], \tag{12}$$

and use the diagonalization $\tilde{\boldsymbol{\Gamma}} = diag(\boldsymbol{\Gamma})$.

The update rules for the parameters of the variational distributions are found applying Equation 10, where the optimal parameter is denoted using the superscript $\star$. For details on the derivation, see Appendix A.

The parameters of the i-th node $\mathbf{A}^i$ are updated according to:

$$\boldsymbol{\Sigma}^{i\star} = (\mathbb{E}[\tau] \sum_{s=1}^{S} \hat{\mathbf{P}}^i \mathbf{X}_s^{\Sigma P} + \tilde{\boldsymbol{\Gamma}}^{E(i)})^{-1}, \qquad \boldsymbol{\mu}^{i\star} = ~~\mathbb{E}[\tau]~~\mathbb{E}[\tau]\boldsymbol{\Sigma}^{i\star} \sum_{s=1}^{S} y_s \hat{\boldsymbol{\mu}}^i \mathbf{X}_s^{\mu}. \tag{13}$$

The update rules for the parameters of edge j are:

$$\boldsymbol{\alpha}_j^{q\star} = \boldsymbol{\alpha}_j^p + \frac{1}{2} \sum_{i \in N(j)} D_{ij}, \qquad \boldsymbol{\beta}_j^{q\star} = \boldsymbol{\beta}_j^p + \frac{1}{2} \sum_{i \in N(j)} tr_{E(i)/j}[\mathbf{P}^i \tilde{\boldsymbol{\Gamma}}^{E(i)/j}]_{\boldsymbol{e}^j, \boldsymbol{e}^j}, \tag{14}$$

where $D_{ij} = \prod_{\boldsymbol{e}^k \in E(i) \wedge \boldsymbol{e}^k \neq \boldsymbol{e}^j} \dim(\boldsymbol{e}^k)$ is the product of dimensions of the edge of node i except the j-th edge. While $tr_{E(i)/j}$ is the partial trace, tracing the tensor over all the edges of node i, except the j-th edge. Note

that the result of the partial trace $tr_{\bar{E}(i)/j} \in V_j \otimes V_j$. Consequently, we use the subscript $\{\boldsymbol{e}^j, \boldsymbol{e}^j\}$ to indicate that we select the diagonal elements, the only non zero elements due to the diagonal structure of $\tilde{\mathbf{\Gamma}}$.

The update rules for the parameters of the noise precision τ are:

$$\alpha_\tau^{q\star} = \alpha_\tau^p + \frac{S}{2}, \qquad \beta_\tau^{q\star} = \beta_\tau^p + \frac{1}{2}\sum_{s=1}^{S}(\|y_s - \langle \boldsymbol{\mu}\mathbf{X}_s^\mu \rangle\|^2 + \langle (\mathbf{P}_{\sim} - \boldsymbol{\mu}_{\sim} \otimes \boldsymbol{\mu}_{\sim})\mathbf{X}^{\Sigma P}_{\sim s} \rangle). \tag{15}$$

The equations for the parameter updates derived in this section, are iteratively applied until reaching desired convergence. The ELBO will be strictly increasing after each update and is guaranteed to reach a local optimum.

One of the main attractive qualities, distinguishing probabilistic methods from standard ones, is the ability to derive the uncertainty associated with the pointwise estimation. Pointwise estimates of means of the posterior distributions are directly derived from the tensor networks $\boldsymbol{\mu}$ by $\mu_s = \langle \boldsymbol{\mu}\mathbf{X}_s^\mu \rangle$.

The estimate of the variance for the BTNR models, σ_s^2 for sample s, is given by the sum of the epistemic and aleatoric variance. The epistemic variance σ_{Es}^2 represents the uncertainty on the model parameters while the aleatoric variance represents the intrinsic estimated noise in the data σ_A^2. They can be derived from the BTNR method as

$$\sigma_{Es}^2 = \langle (\mathbf{P}_{\sim} - \boldsymbol{\mu}_{\sim} \otimes \boldsymbol{\mu}_{\sim})\mathbf{X}^{\Sigma P}_{\sim s} \rangle, \qquad \sigma_A^2 = \mathbb{E}[\tau]\mathbb{E}[\tau^{-1}]^{-1}, \qquad \sigma_s^2 = \sigma_{Es}^2 + \sigma_A^2. \tag{16}$$

We can interestingly notice that the aleatoric variance is strictly related to the mean epistemic variance and the MSE mean squared error (MSE) of the point estimate prediction by evaluating σ_A^2 through the update equations of τ (15). The contributing elements of Equation 15 can be rewritten as

$$\sum_{s=1}^{S}\|y_s - \langle \boldsymbol{\mu}\mathbf{X}_s^\mu \rangle\|^2 = S \cdot \text{MSE} \quad \text{and} \quad \sum_{s=1}^{S}\langle (\mathbf{P}_{\sim} - \boldsymbol{\mu}_{\sim} \otimes \boldsymbol{\mu}_{\sim})\mathbf{X}^{\Sigma P}_{\sim s} \rangle = \sum_{s=1}^{S}\sigma_{Es}^2 = S \cdot \bar{\sigma}_E^2,$$

where $\bar{\sigma}_E^2$ is the mean epistemic variance representing the uncertainty on the model parameters and MSE is the mean squared error. Finally approximating the aleatoric variance as

$$\sigma_A^2 = \mathbb{E}[\tau]\mathbb{E}[\tau^{-1}]^{-1} = \frac{\beta_\tau^{q\star}}{\alpha_\tau^{q\star}}\frac{\beta_\tau^{q\star}}{\alpha_\tau^{q\star} - 1} = \frac{\beta_\tau^p + \frac{S}{2}(MSE + \bar{\sigma}_E^2)}{\alpha_\tau^p + \frac{S}{2}}\frac{\beta_\tau^p + \frac{S}{2}(MSE + \bar{\sigma}_E^2)}{\alpha_\tau^p + \frac{S}{2} - 1} \simeq \frac{2}{S}\beta_\tau^p + (MSE + \bar{\sigma}_E^2). \tag{17}$$

where we assumed in the last step $2\alpha_\tau^p \ll S$ assumed that $2(\alpha_\tau^p - 1) \ll S$.

It is important to provide metrics and experiments able to evaluate the uncertainty estimation quality of our method, at least for the selected structures. The metrics we will use are sharpness (SH) and Expected Calibration Error (ECE) (Kuleshov et al., 2018). Sharpness SH is a measure of how confident is the model in average about its pointwise estimates, calculated by $\text{SH} = S^{-1}\sum_s^S \sigma_s$, where S is the number of samples.

ECE evaluates if the uncertainty provided by the model, can properly distribute the predictions in the expected quantiles. Meaning that, as in our assumption of the regression problem in Equation 2, the labels are distributed as $y_s \sim \mathcal{N}(\mu_s, \sigma_s^2)$ and we should have constant fraction of data under intervals proportional to the standard deviation. Given a set of quantiles $P = \{0.1, 0.2, \ldots, 1.0\}$, and defining as $O(p)$ the model observed fraction of the data in quantile p, ECE is derived as

$$\text{ECE} = \frac{1}{|P|}\sum_{p \in P}\big|\, O(p) - p \,\big|, \qquad O(p) = \frac{1}{S}\sum_{s=1}^{S}\mathbf{1}[\, y_s \leq \mu_s + \sigma_s\, \Phi^{-1}(p)\,], \tag{18}$$

where Φ^{-1} is the standard-normal quantile function, $|P|$ is the number of chosen quantiles and $\mathbf{1}$ is the indicator function. A value near zero means the model is well calibrated.

4.3 Mapping to alternating least squares

The updates in Equation 13 for the nodes based on the Bayesian inference method can trivially include, with slight modifications, the procedure to solve a standard ALS algorithm with Ridge like regularization. The $\boldsymbol{\mu}$ network will now represent the tensor of the coefficients to learn, the $\boldsymbol{\Sigma}$ network is set to zero and $\mathbf{P}^i$ in equation equation 11 will simply be the outer product of the $\boldsymbol{\mu}^i$ node. The environment $\hat{\mathbf{P}}^i$ is obtained by removing from the network the $\boldsymbol{\mu}^i$ node.

$$\boldsymbol{\Sigma} = 0 \quad \mathbf{P} = \boldsymbol{\mu}^i \otimes \boldsymbol{\mu}^i \quad \hat{\mathbf{P}}^i = \hat{\boldsymbol{\mu}}^i \otimes \hat{\boldsymbol{\mu}}^i. \tag{19}$$

The contraction $\hat{\mathbf{P}}^i \mathbf{X}_s^{\Sigma}$ represents the second derivative of the mean squared error loss with respect to the node $\boldsymbol{\mu}^i$. The edge parameters are not updated according to the Bayesian equation, but a priori defined or selected through hyper-parameter optimization. For a Ridge penalty, the edge parameters would all have the same value (i.e. the regularization strength). The $\tilde{\boldsymbol{\Gamma}}$ tensor will be of constant value and uniform defined by the prior. Since $\tilde{\boldsymbol{\Gamma}}$ derives from the tensor product of the expectation of the edges, to obtain an algorithmically equivalent procedure to Ridge regularization, the regularization parameters λ for each node should be defined as $\lambda^{1/\dim(E(\text{node}))}$ to ensure uniformity across all nodes.

4.4 Algorithmic specifications

A great computational advantage of using Bayesian methods during learning, is that the hyper-parameter selection for bond dimension and regularization, is embedded in the method. As for tensor networks this means that the dimension and the relevance of the edges is learned during training. Experiment wise, automatic relevance determination allows to avoid the ablation study over the extensive hyper-parameters space, which include all possible combinationsof edge dimensions below a fixed threshold. Usually in traditional tensor network learning, the ablation happens with a fixed dimension of the edges and during the ablation all the edges have uniform dimensions. Although it is usually enough, it is a clear limitation in the space of the models the ablation study can cover.

The relevance of the edges is embedded in the learning through Equation 13, by the term $\tilde{\boldsymbol{\Gamma}}^{E(i)}$, which contains the product of expectation of the edges related to node i. Intuitively, the greater the expectation of one parameter associated to an edge, the more that dimension will be regularized (pushed to zero). This automatically cancel the contribution of a dimension during learning.

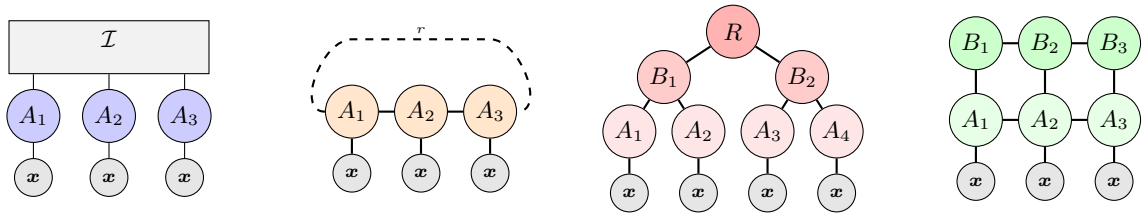
In applications, we might want to also computationally remove the edges that are highly regularized, to accelerate the calculations. Therefore, we define a minimal contribution threshold t to trim the edge dimensions associated to a large expectation value of their related λ. Effectively for any edge i and dimension j, given a threshold t the trimming criterion is explicitly removing dimension j if $\mathbb{E}_q[\lambda_{jk}] > t$.

The introduction of the trimming also requires a choice on how often the trim is performed. During experiments we choose a small number of swipes as warmups, where posterior distribution is not updated and we impose a fixed number of swipes in between trims.

Although ARD offers automatic regularization, instability can still manifest in early iterations, where initialization can often influence if the model will even start to learn. The hyper-parameters we use to regulate initial stability are the priors of the edges, which define initial regularization and initialization strength. Initialization strength is a scalar multiplied to the randomly initialized node (using a standardized normal distribution). For simplicity we define two regimes, one strongly regularized and one weakly regularized, that will be associated to a model dataset during ablation.

5 Experiental Specifications

In the following we detail the experimental setup and algorithmic choices used to produce the tables and plots of this work. Additional details on the experiments are reported in Appendix D.



(a) CPD (b) MPS/TR (c) BTT (d) LMPO2

Figure 3: Diagrammatic representation of the structures examined in this paper. The $\mathcal{I}$ node represents the hyper-diagonal tensor of ones. In (b) when the dimension of the bond r is one, it is represented as a matrix product state, while when it is greater than one, it is represented as a tensor ring.

Table 1: Test R^2 ($\times 100$) at minimum validation loss, reported as mean $\pm$ std over seeds. Best result per dataset in bold.

Model	AB	AI	AP	BK	CO	EE	OB	RS	SB	SP
B-CPD	59.5 ± 0.9	40.9 ± 0.7	38.3 ± 1.9	66.7 ± 0.1	85.7 ± 1.5	99.7 ± 0.0	65.5 ± 14.2	80.6 ± 2.1	62.6 ± 1.0	20.6 ± 0.4
B-LMPO2	59.1 ± 1.2	32.8 ± 0.4	**45.8** ± 0.7	66.1 ± 0.3	83.9 ± 2.0	99.1 ± 0.2	62.1 ± 3.0	76.4 ± 4.0	67.4 ± 2.6	4.7 ± 25.4
B-MPS	59.5 ± 0.4	40.3 ± 0.9	43.2 ± 1.5	67.0 ± 0.0	85.3 ± 1.1	99.5 ± 0.1	54.0 ± 16.6	73.7 ± 1.8	66.6 ± 1.2	20.2 ± 0.1
B-BTT	56.3 ± 2.1	32.4 ± 0.4	42.9 ± 1.2	66.1 ± 0.4	81.9 ± 0.7	99.3 ± 0.2	62.4 ± 4.0	78.7 ± 4.6	67.3 ± 2.2	19.4 ± 1.7
B-TR	59.8 ± 0.5	39.2 ± 0.3	44.2 ± 4.0	66.6 ± 0.1	81.6 ± 0.3	99.4 ± 0.2	62.7 ± 20.8	76.4 ± 2.4	70.5 ± 0.5	19.7 ± 0.1
A-CPD	59.3 ± 1.4	42.0 ± 0.7	37.2 ± 3.0	66.2 ± 0.3	85.8 ± 1.4	99.7 ± 0.0	72.7 ± 1.3	81.1 ± 2.2	51.6 ± 9.7	13.0 ± 3.1
A-LMPO2	59.4 ± 0.4	37.8 ± 3.1	36.2 ± 1.8	65.6 ± 0.5	83.6 ± 3.5	**99.8** ± 0.0	71.5 ± 1.1	78.0 ± 7.6	49.9 ± 3.3	-15.2 ± 22.9
A-MPS	58.3 ± 2.1	40.3 ± 0.4	35.3 ± 2.6	65.9 ± 0.2	84.2 ± 2.4	**99.8** ± 0.0	69.8 ± 2.0	64.2 ± 11.6	39.1 ± 3.9	-4.0 ± 13.7
A-BTT	58.2 ± 3.8	37.7 ± 0.6	33.0 ± 2.6	66.4 ± 0.2	82.9 ± 2.3	**99.8** ± 0.0	67.5 ± 5.0	81.3 ± 2.1	-38.6 ± 141.6	0.1 ± 14.8
A-TR	58.6 ± 1.2	40.1 ± 0.5	37.0 ± 1.6	65.4 ± 0.5	85.1 ± 1.0	**99.8** ± 0.0	72.5 ± 1.0	77.6 ± 5.1	37.1 ± 2.7	11.3 ± 1.2
BDE	**60.6** ± 0.4	3.2 ± 2.4	26.9 ± 1.1	**94.3** ± 0.1	**90.4** ± 0.8	**99.8** ± 0.0	86.1 ± 1.9	**83.7** ± 0.4	**79.3** ± 0.3	10.7 ± 4.8
HSBNN	58.1 ± 1.0	**51.3** ± 0.4	24.7 ± 0.6	81.2 ± 3.4	78.3 ± 0.9	92.2 ± 0.3	77.3 ± 2.6	76.8 ± 0.9	67.7 ± 1.3	**21.9** ± 0.8
BASS	57.2 ± 0.9	47.4 ± 2.0	24.7 ± 0.9	88.7 ± 0.5	85.2 ± 1.0	99.7 ± 0.0	62.4 ± 2.4	77.3 ± 0.8	75.7 ± 0.3	14.3 ± 1.8
GP	57.3 ± 0.3	46.7 ± 0.3	18.3 ± 0.3	82.4 ± 0.5	80.3 ± 0.0	95.1 ± 0.0	**87.6** ± 0.3	74.9 ± 0.0	32.4 ± 0.5	-0.9 ± 0.0

We investigate the performance of the proposed Baeysian learning methods contrasting our framework with the corresponding ALS procedure as described in Section 4 as well as different non tensor network Bayesian baselines, which will be described in the following.

As a feature map we chose the polynomial feature map described in Equation 3 due to the well studied theoretical expressivity of representation. For tensor network models we contrast our learning procedure

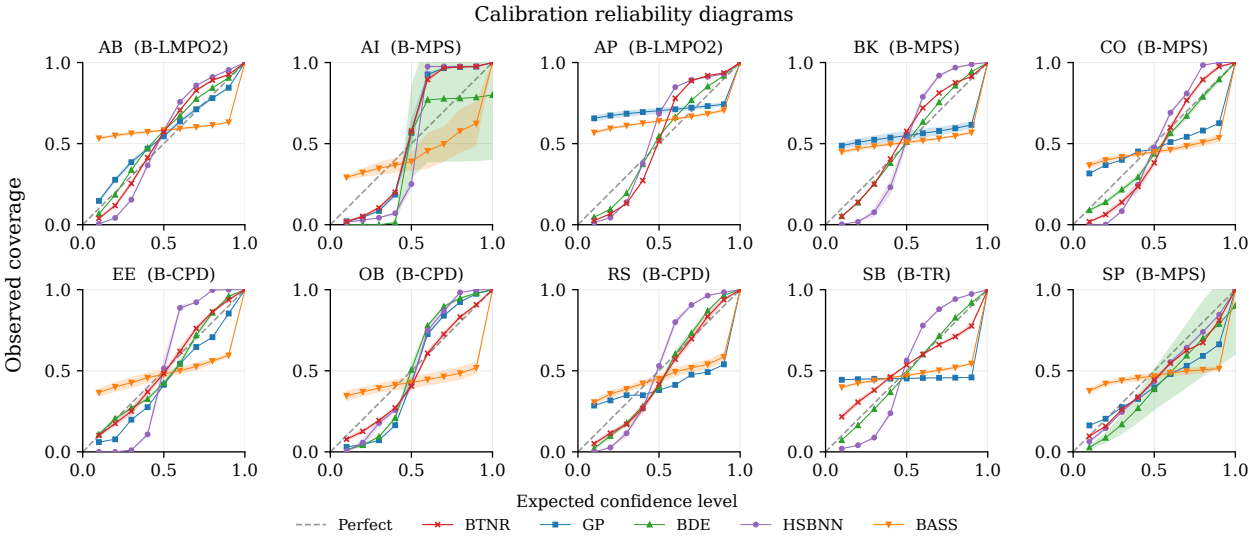


Figure 4: Reliability diagrams. Observed coverage against expected quantile level for each model. The dashed diagonal marks perfect calibration. Curves below it are over-confident and curves above it are under-confident.

with the commonly used ALS procedure as reported in Equation 19. We analyze four different tensor network structures, chosen due to their widespread use in the literature and different latent topologies.

The structures we consider are the CPD (Ayvaz and De Lathauwer, 2022), Matrix Product States/ Tensor Train (MPS/TT) (Schollwöck, 2011; Stoudenmire and Schwab, 2016; Batselier et al., 2019), linear MPO^2 (LMPO2) (Ciolli et al., 2026) as well as the Binary ~~tensor tree~~ Tensor Tree (BTT) (Cheng et al., 2019) and Tensor Ring (TR) (Zhao et al., 2016), not previously considered in this context. ~~We~~ Topologically, the TR is a renown tensor network structure presenting loops in its topology. We included it in the study to highlight the method's effectiveness also on often problematic loop structures. We provide a graphical representation of the structures in Figure 3.

As for the chosen baselines, we consider Gaussian Processes (GP) (MacKay et al., 1998), Bayesian Deep Ensembles (BDE) using the code provided by Sommer et al. (2025), Horse Shoe Bayesian Neural Networks (HSBNN) using the code reported in Overweg et al. (2019), Bayesian Adaptive Smoothing Splines, a python implementation of Friedman (1991) retrievable in the public repository pyBASS (Los Alamos National Laboratory, 2021). We furthermore describe the baselines and their ablation in Appendix D.

All experimentations in the paper are performed with the same splitting of the data, the datasets are divided in train, validation and test set with a rapport of $70:15:15$ respectively. The data is standardized respect to the train set. We first perform an ablation study, training on the train set and choosing the hyperparameter configuration that provide the best mean R^2 over five seeds. Afterwards the hyperparameters are selected we run for ten different seeds the training and evaluation of the model. The reported results are evaluated on the test set at lowest validation loss.

Different models and methods have different hyper-parameters, detailed in Appendix D. In general, tensor networks models hyper-parameter search, validate over the degree of the polynomial map L and the strong versus weak regularization scheme, for both BTNR and ALS. Additionally for ALS we iterate over different values of the edge dimension, imposing the dimension of all edges not contracting with the input to be uniform.

We report a summarized description of the datasets in the Appendix D, Table 7. The code to run the tests and reproduce images and tables can be found at Anonymous Repository (2026)

6 Results and Discussion

In this section we evaluate the quality of the point estimates contrasted it with classical tensor network learning methods. Furthermore we analyse the quality of the estimated uncertainty by reporting the calibration curves of the model, as well as a study on the confidence, comparing with non tensor network Bayesian learning models.

The first metric we report to estimate the quality of the learning is the R^2 of the model predictions. In Table 1 we report the averaged R^2 with errors over the test set. We report with the prefix B- the model learned using BTNR, while with A- the model learned using ALS.

Table 1 seems to suggests that point-estimate predictions of the BTNR method can obtain competitive results with respect to the ALS methods. While in some dataset we can see better or similar performance when compared to baselines. Notably, the TR, chosen as representative of loop structures, performs well compared to loopless structures. ALS method should theoretically ablate over the full hyper-parameter space of edge dimension and regularization to obtain best generalization results. With same structure and feature map basis, we can see that the BTNR method in general upper bounds the ALS R^2, intuitively indicating that the model inferred by BTNR has automatically and without need for extensive validation reached the parameters that best generalize.

As for uncertainty estimation we report for each dataset, a representative chosen between BTNR learned tensor networks and compare it against the baselines. We choose to report the best performing structure ~~for each dataset in the specified metric~~as obtained considering the pointwise estimated mean prediction on the validation set, for clarity of visualization and to showcase the effectiveness of the BTRN method, which is untied from the model structure.

In Figure 4 we report the reliability curves for each dataset, where on the y-coordinate we report the model observed quantile and in the x-coordinate the effective quantile.

In Figure 5 we report a table aiming at summarizing the average over datasets sharpness and calibariation.

Due to be the arbitrary choice of normalization and an average over different datasets the image is to be regarded as a qualitative and easy to read intuitive evaluation, the per dataset plot is reported in Appendix C in Figure 7. For each dataset we divide the sharpness of each model by the real standard deviation of the labels of the dataset, then for each model we average over the datasets. From Figure 4 and Figure 5 we can infer that the uncertainty estimation are well calibrated in respect to the chosen baseline, and seem to evaluate the uncertainty on the data with favorable precision when considering the ECE metric whereas GP and BASS are observed to have more sharpness.

We provide further explanations and experiments in the Appendix. In Appendix A we go through the details of the derivation of the update equations presented in Section 4. In Appendix C we provide additional metrics and experiments to evaluate the quality of the estimated uncertainties. In Appendix D we provide additional details on the experiments, especially on the dataset and the ablation study that led to Table 1.

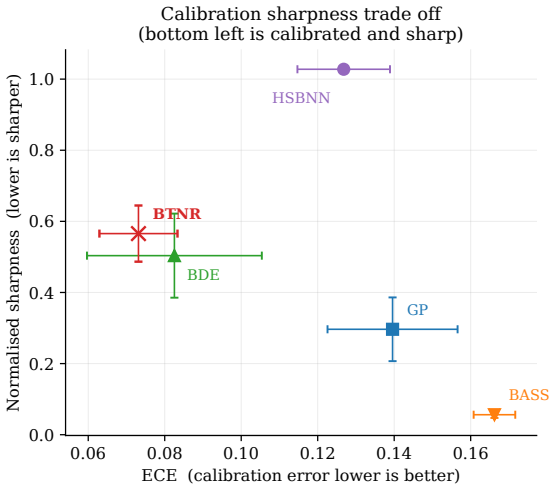


Figure 5: Calibration and sharpness trade off. Each marker places a model at its expected calibration error and its normalised sharpness, the bottom left corner is the desirable combination of low calibration error and high confidence.

7 Conclusions

In this paper we presented the Bayesian Tensor Network Regression (BTNR) framework and derived a procedure to learn generic tensor network structures using Bayesian learning. Using our framework we implemented a series of prominent tensor network structures used for multivariate polynomial regression including the CPD, TT/MPS, and MPO^2 formats and to highligtht the versatility we also included the BTT ~~that has~~ and TR that have not previously been explored in this context. We leveraged the existing tensor network module quimb (Gray, 2018) to accommodate BTNR update equations for the weights to simply implement and abstract tensor network structures. We contrasted these procedures to their corresponding non Bayesian standard ridge regularized ALS implementation as well as various non tensor network based Bayesian models. Notably, the BTNR implements automatic relevance determination and thereby can efficiently prune irrelevant structures as opposed to conventional non-Bayesian estimation in which paratemers specifying the tensor network complexity has to be exhaustively evaluated. Despite careful tuning of the ALS procedure we observed that the BTNR procedure in general produces on par results providing an efficient and robust inference framework for tensor network based multivariate polynomial regression. We further evaluated the quality of the uncertainty estimates against prominent Bayesian regression modeling procedures and found that BTNR provided favorable uncertainty estimates. The developed BTNR framework is generic and we hope that the approach can provide a foundation for future developments of new tensor network topologies for regression based on Bayesian inference.

The presented framework can highly benefit from proper, tensor networks aimed initialization techniques as Guo and Draper (2026) emphasizes, as well as exploring methods to best initialize the priors. Additionally, the extension to multidimensional outputs, using Wishart distribution to model the precision of the likelihood. Furthermore, we presently restrict the analyses to a polynomial feature map. Future studies should explore other feature maps and we expect this choice to be problem dependent and also influence the predictive performance and uncertainty estimates.

Limitations A relevant limitation of the presented framework is the non convexity of the proposed solution to the regression problem, leading to the lack of guarantees to reach the optimal ELBO. Consequently, we lose also guarantees of reaching optimal model structure through pruning. Additionally we have considered an arbitrary set of distributions to describe the model, while other probabilistic families could be explored such as discussed in Cheng et al. (2022).

References

bde. URL https://github.com/scikit-learn-contrib/bde.

Anonymous Repository. Btnr, 2026. URL https://github.com/0AnonymousSubmission/tmlr_btnr_2026.

Muzaffer Ayvaz and Lieven De Lathauwer. CPD-Structured Multivariate Polynomial Optimization. *Frontiers in Applied Mathematics and Statistics*, 8, 3 2022. ISSN 2297-4687. doi: 10.3389/fams.2022.836433. URL https://doi.org/10.3389%2Ffams.2022.836433.

Kim Batselier. A khatri-rao product tensor network for efficient symmetric mimo volterra identification. *IFAC-PapersOnLine*, 56(2):7282–7287, 2023. ISSN 2405-8963. doi: https://doi.org/10.1016/j.ifacol.2023.10.339. URL https://www.sciencedirect.com/science/article/pii/S2405896323007061. 22nd IFAC World Congress.

Kim Batselier, Zhongming Chen, and Ngai Wong. Tensor Network alternating linear scheme for MIMO Volterra system identification. *eprint*, 7 2016. URL https://arxiv.org/abs/1607.00127v2.

Kim Batselier, Ching-Yun Ko, and Ngai Wong. Extended kalman filtering with low-rank tensor networks for mimo volterra system identification. In *2019 IEEE 58th Conference on Decision and Control (CDC)*, pages 7148–7153, 2019. doi: 10.1109/cdc40024.2019.9028895.

David M. Blei, Alp Kucukelbir, and Jon D. McAuliffe. Variational inference: A review for statisticians. *Journal of the American Statistical Association*, 112(518):859–877, 2017. doi: 10.1080/01621459.2017.1285773. URL https://doi.org/10.1080/01621459.2017.1285773.

Lei Cheng, Feng Yin, Sergios Theodoridis, Sotirios Chatzis, and Tsung-Hui Chang. Rethinking bayesian learning for data analysis: The art of prior and inference in sparsity-aware modeling. *IEEE Signal Processing Magazine*, 39(6):18–52, 2022. doi: 10.1109/msp.2022.3198201.

Song Cheng, Lei Wang, Tao Xiang, and Pan Zhang. Tree tensor networks for generative modeling. *Phys. Rev. B*, 99:155131, Apr 2019. doi: 10.1103/PhysRevB.99.155131. URL https://link.aps.org/doi/10.1103/PhysRevB.99.155131.

Niccolò Ciolli, Anders Vestergaard Nørskov, Michael Kastoryano, Petr Taborsky, and Morten Mørup. (mpo): Multivariate polynomial optimization based on matrix product operators, 2026. URL https://arxiv.org/abs/2607.15916.

J. Ignacio Cirac, David Pérez-García, Norbert Schuch, and Frank Verstraete. Matrix product states and projected entangled pair states: Concepts, symmetries, theorems. *Rev. Mod. Phys.*, 93:045003, Dec 2021. doi: 10.1103/RevModPhys.93.045003. URL https://link.aps.org/doi/10.1103/RevModPhys.93.045003.

Jerome H. Friedman. Multivariate adaptive regression splines. *The Annals of Statistics*, 19(1):1–67, 1991. ISSN 00905364, 21688966. URL http://www.jstor.org/stable/2241837.

Yarin Gal and Zoubin Ghahramani. Dropout as a bayesian approximation: Representing model uncertainty in deep learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, volume 48 of *Pmlr*, pages 1050–1059, 2016.

Jacob R Gardner, Geoff Pleiss, David Bindel, Kilian Q Weinberger, and Andrew Gordon Wilson. Gpytorch: Blackbox matrix-matrix gaussian process inference with gpu acceleration. In *Advances in Neural Information Processing Systems*, 2018.

Tilmann Gneiting and Adrian E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378, 2007.

Michael Götte, Reinhold Schneider, and Philipp Trunschke. A block-sparse tensor train format for sample-efficient high-dimensional polynomial regression, 2021. URL https://arxiv.org/abs/2104.14255.

Nithin Govindarajan, Nico Vervliet, and Lieven De Lathauwer. Regression and classification with spline-based separable expansions. *Frontiers in big Data*, 5:688496, 2022.

Lars Grasedyck, Melanie Kluge, and Sebastian Krämer. Alternating least squares tensor completion in the tt-format. *SIAM Journal on Scientific Computing*, 2019. URL https://arxiv.org/abs/1509.00311. arXiv:1509.00311.

Johnnie Gray. quimb: a python library for quantum information and many-body calculations. *Journal of Open Source Software*, 3(29):819, 2018. doi: 10.21105/joss.00819.

Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, volume 70 of *Pmlr*, pages 1321–1330, 2017.

Erdong Guo and David Draper. A bayesian approach to tensor networks. *Neurocomputing*, 675:132961, April 2026. ISSN 0925-2312. doi: 10.1016/j.neucom.2026.132961. URL http://dx.doi.org/10.1016/j.neucom.2026.132961.

Stijn Hendrikx, Martijn Boussé, Nico Vervliet, and Lieven De Lathauwer. Algebraic and optimization based algorithms for multivariate regression using symmetric tensor decomposition. In *2019 IEEE 8th International Workshop on Computational Advances in Multi-Sensor Adaptive Processing (CAMSAP)*, pages 475–479. Ieee, 2019.

Dan Hendrycks and Kevin Gimpel. A baseline for detecting misclassified and out-of-distribution examples in neural networks. In *International Conference on Learning Representations (ICLR)*, 2017.

Jesper L. Hinrich and Morten Mørup. Probabilistic tensor train decomposition. In *European Signal Processing Conference*, volume 2019-September, 2019. doi: 10.23919/eusipco.2019.8903177.

Jesper L Hinrich, Kristoffer H Madsen, and Morten Mørup. The probabilistic tensor decomposition toolbox. *Machine Learning: Science and Technology*, 1(2):025011, jun 2020. doi: 10.1088/2632-2153/ab8241. URL `https://dx.doi.org/10.1088/2632-2153/ab8241`.

Sebastian Holtz, Thorsten Rohwedder, and Reinhold Schneider. The Alternating Linear Scheme for Tensor Optimization in the Tensor Train Format. *SIAM Journal on Scientific Computing*, 34(2):A683–a713, 1 2012. doi: 10.1137/100818893. URL `https://doi.org/10.1137%2F100818893`.

Eddy Ilg, Özgün Çiçek, Silvio Galesso, Aaron Klein, Osama Makansi, Frank Hutter, and Thomas Brox. Uncertainty estimates and multi-hypotheses networks for optical flow. In *Computer Vision – ECCV 2018*, volume 11211 of *Lecture Notes in Computer Science*, pages 677–693, 2018.

Alex Kendall and Yarin Gal. What uncertainties do we need in bayesian deep learning for computer vision? In *Advances in Neural Information Processing Systems*, volume 30, pages 5574–5584, 2017.

Afra Kilic and Kim Batselier. Interpretable Bayesian Tensor Network Kernel Machines with Automatic Rank and Feature Selection. 1 2025. URL `https://arxiv.org/abs/2507.11136`.

Kriton Konstantinidis, Yao Lei Xu, Qibin Zhao, and Danilo P. Mandic. Variational bayesian tensor networks with structured posteriors. In *ICASSP 2022 - 2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 3638–3642, 2022. doi: 10.1109/icassp43922.2022.9747861.

Volodymyr Kuleshov, Nathan Fenner, and Stefano Ermon. Accurate uncertainties for deep learning using calibrated regression. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80 of *Pmlr*, pages 2801–2809, 2018.

Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems*, volume 30, pages 6402–6413, 2017.

Los Alamos National Laboratory. pybass, 2021. URL `https://github.com/lanl/pyBASS`.

David JC MacKay et al. Introduction to gaussian processes. *NATO ASI series F computer and systems sciences*, 168:133–166, 1998.

James E. Matheson and Robert L. Winkler. Scoring rules for continuous probability distributions. *Management Science*, 22(10):1087–1096, 1976.

Microsoft. horseshoe-bnn. URL `https://github.com/microsoft/horseshoe-bnn`.

Morten Mørup and Lars Kai Hansen. Automatic relevance determination for multi-way models. *Journal of Chemometrics*, 23, 2009. ISSN 1099128x. doi: 10.1002/cem.1223.

Hiske Overweg, Anna-Lena Popkes, Ari Ercole, Yingzhen Li, José Miguel Hernández-Lobato, Yordan Zaykov, and Cheng Zhang. Interpretable outcome prediction with sparse bayesian neural networks in intensive care, 2019. URL `https://arxiv.org/abs/1905.02599`.

Mahdi Pakdaman Naeini, Gregory F. Cooper, and Milos Hauskrecht. Obtaining well calibrated probabilities using bayesian binning. In *Proceedings of the 29th AAAI Conference on Artificial Intelligence (AAAI)*, pages 2901–2907, 2015.

Tim Pearce, Alexandra Brintrup, Mohamed Zaki, and Andy Neely. High-quality prediction intervals for deep learning: A distribution-free, ensembled approach. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80 of *Pmlr*, pages 4072–4081, 2018.

Ulrich Schollwöck. The density-matrix renormalization group in the age of matrix product states. *Annals of Physics*, 326(1):96–192, 2011. ISSN 0003-4916. doi: https://doi.org/10.1016/j.aop.2010.09.012. URL https://www.sciencedirect.com/science/article/pii/S0003491610001752. January 2011 Special Issue.

Václav Šmídl and Anthony Quinn. On bayesian principal component analysis. *Computational statistics & data analysis*, 51(9):4101–4123, 2007.

Emanuel Sommer, Jakob Robnik, Giorgi Nozadze, Uros Seljak, and David Rügamer. Microcanonical langevin ensembles: Advancing the sampling of bayesian neural networks, 2025. URL https://arxiv.org/abs/2502.06335.

Edwin Stoudenmire and David J Schwab. Supervised learning with tensor networks. *Advances in neural information processing systems*, 29, 2016.

Kevin Tran, Willie Neiswanger, Junwoong Yoon, Qingyang Zhang, Eric P. Xing, and Zachary W. Ulissi. Methods for comparing uncertainty quantifications for material property predictions. *Machine Learning: Science and Technology*, 1(2):025006, 2020.

Qibin Zhao, Guoxu Zhou, Shengli Xie, Liqing Zhang, and Andrzej Cichocki. Tensor ring decomposition. *arXiv preprint arXiv:1606.05535*, 2016.

A Mathematical Derivation

We now derive the update equations from the ELBO maximization rules in Equation 10.

Let $\mathbf{A}$ be a tensor network defined by the set of the nodes $\{\mathbf{A}^1, \mathbf{A}^2, \dots, \mathbf{A}^n\}$ and edges $\{e^1, \dots, e^m\}$.

We firstly derive the update equation for the parameters of a single node $\mathbf{A}^i$.

It is obtained by solving

$$\log p(\mathbf{A}^i) = \mathbb{E}_{q(\Theta/\mathbf{A}^i)}[\log p(\boldsymbol{y}, \Theta)]$$
$$\log p(\mathbf{A}^i) = \mathbb{E}_{q(\Theta/\mathbf{A}^i)}[\log p(\boldsymbol{y}, \Theta)]. \tag{20}$$

We explicit explicitly show the two sides of the equation, considering that in noting that considering the right side, we can ignore all the terms that would be constant in with respect to $\mathbf{A}^i$.

$$\mathbb{E}_{q(\Theta/\mathbf{A}^i)}[\log p(\boldsymbol{y}, \Theta)] = \sum_s \mathbb{E}_{q(\Theta/\mathbf{A}^i)}[\log l_s] + \mathbb{E}_{q(\Theta/\mathbf{A}^i)}[\log p(\mathbf{A}^i)]. \tag{21}$$

Note that the arguments of the gaussian Gaussian are kept as tensors, the distribution is the same as if they would be vectorized, calculated using the logarithm of the distribution and reshaped in the original form.

Except for a constant the right side takes the form:

$$\log l_s \simeq -\frac{\tau}{2}(y_s - \langle \mathbf{A}\mathbf{X}_s \rangle)^2 = -\frac{\tau}{2} y_s^2 + \tau y_s \langle \mathbf{A}\mathbf{X}_s \rangle - \frac{\tau}{2}\langle \mathbf{X}_s \mathbf{A} \rangle \langle \mathbf{A}\mathbf{X}_s \rangle \tag{22}$$
$$= -\frac{\tau}{2} y_s^2 + \tau y_s \langle \hat{\mathbf{A}}^i \mathbf{A}^i \mathbf{X}_s \rangle - \frac{\tau}{2}\langle \mathbf{X}_s \mathbf{A}^i \hat{\mathbf{A}}^i \hat{\mathbf{A}}^i \mathbf{A}^i \mathbf{X}_s \rangle, \tag{23}$$
$$\log p(\mathbf{A}^i) = -\frac{1}{2}\mathbf{A}^i \tilde{\mathbf{\Lambda}}^{E(i)} \mathbf{A}^i + \text{constant}., \tag{24}$$

Where where in step 23 separate the tensor network $\mathbf{A} = \hat{\mathbf{A}}^i \mathbf{A}^i$ is separated, as a contraction between the environment of the node and the node $\mathbf{A}^i$ itself. The term $-\frac{\tau}{2} y_s^2$ is a constant and so it can can therefore be removed. For brevity, the expectation will be written $\mathbb{E}$ without the subscript, which is often obvious by the context.

$$\mathbb{E}_{q(\Theta/\mathbf{A}^i)}[\log l_s] = y_s \mathbb{E}[\tau] \langle \mathbb{E}[\hat{\mathbf{A}}^i] \mathbf{A}^i \mathbf{X}_s \rangle - \frac{\mathbb{E}[\tau]}{2} \langle \mathbf{A}^i \mathbf{X}_s \mathbb{E}[\hat{\mathbf{A}}^i \otimes \hat{\mathbf{A}}^i] \mathbf{X}_s \mathbf{A}^i \rangle$$
$$= y_s \mathbb{E}[\tau] \langle \hat{\boldsymbol{\mu}}^i \mathbf{A}^i \mathbf{X}_s \rangle - \frac{\mathbb{E}[\tau]}{2} \langle \mathbf{A}^i \mathbf{X}_s [\tilde{\boldsymbol{\Sigma}}^i - \hat{\boldsymbol{\mu}}^i \otimes \hat{\boldsymbol{\mu}}^i] \mathbf{X}_s \mathbf{A}^i \rangle$$
$$\mathbb{E}_{q(\Theta/\mathbf{A}^i)}[\log p(\mathbf{A}^i)] = -\frac{1}{2}\mathbf{A}^i \mathbb{E}[\tilde{\mathbf{\Lambda}}^{E(i)}] \mathbf{A}^i = -\frac{1}{2}\mathbf{A}^i \tilde{\mathbf{\Gamma}}^{E(i)} \mathbf{A}^i$$

$$\mathbb{E}_{q(\Theta/\mathbf{A}^i)}[\log l_s] = y_s \mathbb{E}[\tau] \langle \mathbb{E}[\hat{\mathbf{A}}^i] \mathbf{A}^i \mathbf{X}_s \rangle - \frac{\mathbb{E}[\tau]}{2} \langle \mathbf{A}^i \mathbf{X}_s \mathbb{E}[\hat{\mathbf{A}}^i \otimes \hat{\mathbf{A}}^i] \mathbf{X}_s \mathbf{A}^i \rangle \tag{25}$$
$$= y_s \mathbb{E}[\tau] \langle \hat{\boldsymbol{\mu}}^i \mathbf{A}^i \mathbf{X}_s \rangle - \frac{\mathbb{E}[\tau]}{2} \langle \mathbf{A}^i \mathbf{X}_s \hat{\mathbf{P}}^i \mathbf{X}_s \mathbf{A}^i \rangle, \tag{26}$$
$$\mathbb{E}_{q(\Theta/\mathbf{A}^i)}[\log p(\mathbf{A}^i)] = -\frac{1}{2}\mathbf{A}^i \mathbb{E}[\tilde{\mathbf{\Lambda}}^{E(i)}] \mathbf{A}^i = -\frac{1}{2}\mathbf{A}^i \tilde{\mathbf{\Gamma}}^{E(i)} \mathbf{A}^i, \tag{27}$$

Where where we used the definition of $\mathbf{\Gamma}$ in Equation 12 and use the diagonalization $\tilde{\mathbf{\Gamma}} = diag(\mathbf{\Gamma})$.

Finally

$$\sum_s \mathbb{E}[\log l_s] + \mathbb{E}[\log p(\mathbf{A}^i)] =$$
$$= \sum_s y_s \mathbb{E}[\tau] \langle \hat{\boldsymbol{\mu}}^i \mathbf{A}^i \mathbf{X}_s \rangle - \frac{\mathbb{E}[\tau]}{2} \langle \mathbf{A}^i \mathbf{X}_s [\tilde{\boldsymbol{\Sigma}}^i - \hat{\boldsymbol{\mu}}^i \otimes \hat{\boldsymbol{\mu}}^i] \mathbf{X}_s \mathbf{A}^i \rangle - \frac{1}{2} \langle \mathbf{A}^i \tilde{\boldsymbol{\Gamma}}^{E(i)} \mathbf{A}^i \rangle =$$
$$= \{\mathbb{E}[\tau] \sum_s \hat{\boldsymbol{\mu}}^i \mathbf{X}_s\} \mathbf{A}^i - \frac{1}{2} \mathbf{A}^i \{\mathbb{E}[\tau] \sum_s \mathbf{X}_s [\tilde{\boldsymbol{\Sigma}}^i - \hat{\boldsymbol{\mu}}^i \otimes \hat{\boldsymbol{\mu}}^i] \mathbf{X}_s + \tilde{\boldsymbol{\Gamma}}^{E(i)}\} \mathbf{A}^i$$

, we obtain

$$\sum_s \mathbb{E}[\log l_s] + \mathbb{E}[\log p(\mathbf{A}^i)] = \tag{28}$$
$$= \sum_s y_s \mathbb{E}[\tau] \langle \hat{\boldsymbol{\mu}}^i \mathbf{A}^i \mathbf{X}_s \rangle - \frac{\mathbb{E}[\tau]}{2} \langle \mathbf{A}^i \mathbf{X}_s \hat{\mathbf{P}}^i \mathbf{X}_s \mathbf{A}^i \rangle - \frac{1}{2} \langle \mathbf{A}^i \tilde{\boldsymbol{\Gamma}}^{E(i)} \mathbf{A}^i \rangle = \tag{29}$$
$$= \{\mathbb{E}[\tau] \sum_s y_s \hat{\boldsymbol{\mu}}^i \mathbf{X}_s\} \mathbf{A}^i - \frac{1}{2} \mathbf{A}^i \{\mathbb{E}[\tau] \sum_s \mathbf{X}_s \hat{\mathbf{P}}^i \mathbf{X}_s + \tilde{\boldsymbol{\Gamma}}^{E(i)}\} \mathbf{A}^i. \tag{30}$$

If we consider a generic ~~gaussian~~ Gaussian density distribution of $\mathbf{A}^i$, we can notice that the log of the density has the form $\{{\boldsymbol{\Sigma}^i}^{-1} \boldsymbol{\mu}^i\} \mathbf{A}^i - \frac{1}{2} \mathbf{A}^i \{{\boldsymbol{\Sigma}^i}^{-1}\} \mathbf{A}^i$. Obtaining the updates in Equation 13.

For the edges ~~instead we~~ we instead need to repeat the same process, substituting the edge probability distribution and solving the equation:

$$\log p(\boldsymbol{e}^j) = \mathbb{E}_{q(\Theta/\boldsymbol{e}^j)}[\log p(\boldsymbol{y}, \Theta)]$$
$$\log p(\boldsymbol{e}^j) = \mathbb{E}_{q(\Theta/\boldsymbol{\lambda}_j)}[\log p(\boldsymbol{y}, \Theta)]. \tag{31}$$

The terms we consider are

$$\mathbb{E}_{q(\Theta/\boldsymbol{e}^j)}[\log p(\boldsymbol{y}, \Theta)] = \sum_{i \in B(j)} \mathbb{E}[\log p(\mathbf{A}^i)] + \mathbb{E}[\log p(\boldsymbol{e}^j)]$$
$$\mathbb{E}_{q(\Theta/\boldsymbol{\lambda}_j)}[\log p(\boldsymbol{y}, \Theta)] = \sum_{i \in N(j)} \mathbb{E}[\log p(\mathbf{A}^i)] + \mathbb{E}[\log p(\boldsymbol{e}^j)]. \tag{32}$$

Note that $\tilde{\boldsymbol{\Lambda}}^{E(i)}$ is diagonal. The diagonal elements are the parameters associated to the edges of node $\mathbf{A}^i$. The determinant of $\tilde{\boldsymbol{\Lambda}}^{E(i)}$ is then

$$|\tilde{\boldsymbol{\Lambda}}^{E(i)}| = \prod_{\boldsymbol{e}^j \in E(i)} \prod_{k=1}^{\dim(\boldsymbol{e}^j)} \lambda_{jk}^{D_{ij}}, \tag{33}$$

~~Where~~ where $D_{ij} = \prod_{\boldsymbol{e}^k \in E(i) \wedge \boldsymbol{e}^k \neq \boldsymbol{e}^j} \dim(\boldsymbol{e}^k)$.

The logarithm of the prior becomes a series of sums.

$$\log p(\boldsymbol{y}, \Theta) = \log p(\tau) + \sum_s \log l_s + \sum_{i \in nodes} \log p(\mathbf{A}^i) + \sum_{j \in edges} p(\boldsymbol{e}^j)$$

$$\log p(\boldsymbol{y}, \Theta) = \log p(\tau) + \sum_{s} \log l_s + \sum_{i \in \text{nodes}} \log p(\mathbf{A}^i) + \sum_{j \in \text{edges}} p(\boldsymbol{e}^j), \tag{34}$$

where we have

$$\mathbb{E}_{q(\Theta/\boldsymbol{e}^j)}[\log p(\mathbf{A}^i)]_{q(\Theta/\boldsymbol{\lambda}_j)}[\log p(\mathbf{A}^i)] = \tag{35}$$

$$= \frac{1}{2} D_{ij} \sum_{k=1}^{\dim(\boldsymbol{e}^j)} \log \lambda_{jk} - \frac{1}{2} \mathbb{E}[vec(\mathbf{A}^i)^T \tilde{\boldsymbol{\Lambda}}^{E(i)} vec(\mathbf{A}^i)] = \tag{36}$$

$$= D_{ij} \log(\langle \boldsymbol{\lambda}_j \mathbf{1} \rangle) - \frac{1}{2} \mathbb{E}[tr_{E(i)/j}(\mathbf{A}^i \otimes \mathbf{A}^i \tilde{\boldsymbol{\Gamma}}^{E(i)/j})_{\boldsymbol{e}^j \boldsymbol{e}^j}]_j = = D_{ij} \log(\langle \boldsymbol{\lambda}_j \mathbf{1} \rangle) - \frac{1}{2} [\langle tr_{E(i)/j}(\mathbb{E}[\mathbf{A}^i \otimes \mathbf{A}^i] \tilde{\boldsymbol{\Gamma}}^{E(i)/j})_{\boldsymbol{e}^j \boldsymbol{e}^j}]_j \rangle \mathbb{E}_{q(\Theta/\boldsymbol{e}^j)}[\log p(\boldsymbol{e}^j)] \tag{37}$$

As previously, considering

$$\mathbb{E}_{q(\Theta/\boldsymbol{\lambda}_j)}[\log p(\boldsymbol{e}^j)] = \langle \boldsymbol{\alpha}_j^p \log(\boldsymbol{\lambda}_j) \rangle - \langle \mathbf{1} \log(\boldsymbol{\lambda}_j) \rangle - \langle \boldsymbol{\beta}_j^p \boldsymbol{\lambda}_j \rangle. \tag{38}$$

By summing both terms we finally obtain:

$$\sum_{i \in N(j)} \mathbb{E}[\log p(\mathbf{A}^i)] + \mathbb{E}[\log p(\boldsymbol{e}^j)] = \tag{39}$$

$$= \langle \boldsymbol{\alpha}_j^p \log(\boldsymbol{\lambda}_j) \rangle - \langle \boldsymbol{\beta}_j^p \boldsymbol{\lambda}_j \rangle - \langle \mathbf{1} \log(\boldsymbol{\lambda}_j) \rangle - \tag{40}$$

$$- \frac{1}{2} \sum_{i \in N(j)} \langle tr_{E(i)/j}(\mathbb{E}[\mathbf{A}^i \otimes \mathbf{A}^i] \tilde{\boldsymbol{\Gamma}}^{E(i)/j})_{\boldsymbol{e}^j \boldsymbol{e}^j} \boldsymbol{\lambda}_j \rangle + \frac{1}{2} D_{ij} \langle \mathbf{1} \log(\boldsymbol{\lambda}_j) \rangle = \tag{41}$$

$$= \langle \{ \boldsymbol{\alpha}_j^p + \frac{1}{2} \sum_{i \in N(j)} D_{ij} \mathbf{1} - \mathbf{1} \} \log(\boldsymbol{\lambda}_j) \rangle - \langle \{ \boldsymbol{\beta}_j^p + \frac{1}{2} \sum_{i \in N(j)} tr_{E(i)/j}([\boldsymbol{\Sigma}^i + \boldsymbol{\mu}^i \otimes \boldsymbol{\mu}^i] \tilde{\boldsymbol{\Gamma}}^{E(i)/j})_{\boldsymbol{e}^j \boldsymbol{e}^j} \} \boldsymbol{\lambda}_j \rangle. \tag{42}$$

Considering the log form of the gamma distribution:

$$\log(Ga(x, \alpha, \beta)) \sim \{(\alpha - 1)\} \log x - \{\beta\} x$$

$$\log(Ga(x, \alpha, \beta)) \sim \{(\alpha - 1)\} \log x - \{\beta\} x, \tag{43}$$

we can identified identify in vectorized form the new parameters in equation 14.

In line 36, to easily transition present the derivation, we write for a moment the equations in a vectorized form. The passage in line 36 can be better derived if we recall the definition of $\tilde{\boldsymbol{\Lambda}}^{E(i)}$ which is the diagonal matrix, with diagonal the diagonal given by the outer product of the edge parameters vectorized.

$$diag(\tilde{\boldsymbol{\Lambda}}^{E(i)}) = vec(\bigotimes_{\boldsymbol{e}^j \in E(i)} \boldsymbol{\lambda}_j)$$

$$diag(\tilde{\boldsymbol{\Lambda}}^{E(i)}) = vec(\bigotimes_{j \in E(i)} \boldsymbol{\lambda}_j), \tag{44}$$

$$vec(\mathbf{A}^i)^T \tilde{\mathbf{\Lambda}}^{E(i)} vec(\mathbf{A}^i) = tr(vec(\mathbf{A}^i)^T \tilde{\mathbf{\Lambda}}^{E(i)} vec(\mathbf{A}^i)) = tr(vec(\mathbf{A}^i) vec(\mathbf{A}^i)^T \tilde{\mathbf{\Lambda}}^{E(i)}). \tag{45}$$

~~And in tensor form is by explicit the edges indices.~~

~~$vec(\mathbf{A}^i) vec(\mathbf{A}^i)^T \tilde{\mathbf{\Lambda}}^{E(i)} =$~~
~~$= \mathbf{A}^i_{e^j \in \bar{E}(i)} \mathbf{A}^i_{e^{j'} \in \bar{E}(i)} \prod_{e^j \in \bar{E}(i)} (\delta(e^j, e^{j'})) \bigotimes_{e^j \in \bar{E}(i)} \boldsymbol{\lambda}_j = \mathbf{A}^i_{e^j \in \bar{E}(i)} \mathbf{A}^i_{e^{j'} \in \bar{E}(i)} \bigotimes_{e^j \in \bar{E}(i)} diag(\boldsymbol{\lambda}_j)$~~

In order to provide a concrete example of the substitution of the expectation of Equation 45 in line 36 we consider a three dimensional node $\mathbf{A}^i$. We have to explicitly associate edges with indices. We assume that $\bar{E}(i) = \{e^1, e^2, e^3\}$ and we index the edges with the indices:

$$(l, m, n) \rightarrow (e^1, e^2, e^3).$$

~~$\tilde{\mathbf{\Gamma}}^{E(i)} = \bigotimes_{e^j \in \bar{E}(i)} \mathbb{E}[diag(\boldsymbol{\lambda}_j)]$~~

$$tr(vec(\mathbf{A}^i) vec(\mathbf{A}^i)^T \tilde{\mathbf{\Lambda}}^{E(i)}) = \tag{46}$$

$$= \sum_{lmn} \sum_{l'm'n'} \sum_{l''m''n''} \mathbf{A}^i_{lmn} \mathbf{A}^i_{l'm'n'} \delta_{l',l''} \delta_{m',m''} \delta_{n',n''} \lambda_{1l''} \lambda_{2m''} \lambda_{3n''} \delta_{l,l''} \delta_{m,m''} \delta_{n,n''} \tag{47}$$

$$= \sum_{lmn} \sum_{l'm'n'} \mathbf{A}^i_{lmn} \mathbf{A}^i_{l'm'n'} \delta_{l,l'} \delta_{m,m'} \delta_{n,n'} \lambda_{1l} \lambda_{2m} \lambda_{3n}. \tag{48}$$

~~Example with a 3-D node.~~

~~$tr(vec(\mathbf{A}^i) vec(\mathbf{A}^i)^T \tilde{\mathbf{\Lambda}}^{E(i)}) = \sum_{ijki'j'k'} \mathbf{A}^i_{ijk} \delta_{i,i'} \delta_{j,j'} \delta_{k,k'} \lambda_{1i} \lambda_{2j} \lambda_{3k} \mathbf{A}^i_{i'j'k'}$~~
~~$= \sum_{ijki'j'k'} \mathbf{A}^i_{ijk} \delta_{i,i'} \delta_{j,j'} \delta_{k,k'} \lambda_{1i} \lambda_{2j} \mathbf{A}^i_{i'j'k'} \lambda_{3k}$~~
~~$= \sum_k \sum_{k'} \delta_{k,k'} \sum_{iji'j'} \mathbf{A}^i_{ijk} \delta_{i,i'} \delta_{j,j'} \lambda_{1i} \lambda_{2j} \mathbf{A}^i_{i'j'k'} \lambda_{3k}$~~

Remembering that

$$\tilde{\mathbf{\Gamma}}^{E(i)} = \bigotimes_{j \in E(i)} \mathbb{E}[diag(\boldsymbol{\lambda}_j)], \tag{49}$$

~~Now the expectation becomes~~ the expectation with respect to the parameters $\boldsymbol{\lambda}_3$ associated to edge e^3 can be calculated as:

$$\mathbb{E}[tr(vec(\mathbf{A}^i)vec(\mathbf{A}^i)^T\tilde{\mathbf{\Lambda}}^{E(i)})]\mathbb{E}_{q(\Theta/\boldsymbol{\lambda}_3)}[tr(vec(\mathbf{A}^i)vec(\mathbf{A}^i)^T\tilde{\mathbf{\Lambda}}^{E(i)})] = \sum_k\sum_{k'}\delta_{k,k'}\sum_{ijj'i'}\mathbb{E}[\mathbf{A}^i_{ijk}\mathbf{A}^i_{i'j'k'}]\delta_{i,i'}\delta_{j,j'}\mathbb{E}[\lambda_{1i}]\mathbb{E}[\lambda_{2j}]\lambda_{3k}$$

(50)

$$= \mathbb{E}[\sum_{lmn}\sum_{l'm'n'}\mathbf{A}^i_{lmn}\mathbf{A}^i_{l'm'n'}\delta_{l,l'}\delta_{m,m'}\delta_{n,n'}\lambda_{1l}\lambda_{2m}\lambda_{3n}] =$$ (51)

$$= \sum_k\sum_{k'}\sum_{nn'}\delta_{k,k'}\delta_{n,n'}\sum_{ijj'i'}\mathbb{E}[\mathbf{A}^i_{ijk}\mathbf{A}^i_{i'j'k'}]^{\bar{E}(i)/3}\sum_{lm}\sum_{l'm'}\mathbb{E}[\mathbf{A}^i_{lmn}\mathbf{A}^i_{l'm'n'}]\mathbb{E}[\delta_{l,l'}\delta_{m,m'}\lambda_{1l}\lambda_{2m}]\lambda_{3k}\lambda_{3n}$$

(52)

$$= \sum_k\sum_{k'}\sum_{nn'}\delta_{k,k'}\delta_{n,n'}\,tr_{\bar{E}(i)/3E(i)/3}(\mathbb{E}[\mathbf{A}^i\otimes\mathbf{A}^i]^{\bar{E}(i)/3}\mathbb{E}[\mathbf{A}^i\otimes\mathbf{A}^i]\tilde{\mathbf{\Gamma}}^{E(i)/3})_{k,k'nn'}\lambda_{3k}\lambda_{3n} =$$

(53)

$$= \sum_k tr_{\bar{E}(i)/3}(\mathbb{E}[\mathbf{A}^i\otimes\mathbf{A}^i]\tilde{\mathbf{\Gamma}}^{\bar{E}(i)/3})_{k,k}\,tr_{E(i)/3}(\mathbb{E}[\mathbf{A}^i\otimes\mathbf{A}^i]\tilde{\mathbf{\Gamma}}^{E(i)/3})_{nn}\lambda_{3k}\lambda_{3n} =$$

(54)

$$= \langle tr_{\bar{E}(i)/3}(\mathbb{E}[\mathbf{A}^i\otimes\mathbf{A}^i]\tilde{\mathbf{\Gamma}}^{\bar{E}(i)/3})_{\boldsymbol{e}^3,\boldsymbol{e}^3}\langle tr_{E(i)/3}(\mathbb{E}[\mathbf{A}^i\otimes\mathbf{A}^i]\tilde{\mathbf{\Gamma}}^{E(i)/3})_{\boldsymbol{e}^3,\boldsymbol{e}^3}\boldsymbol{\lambda}_3\rangle.$$ (55)

Where we have identified as indexed with $\{\boldsymbol{e}^3,\boldsymbol{e}^3\}$ the diagonal elements of the trace in the edge 3. To be more specific, the result of the trace will be just a bidimensional tensor in the space of $\boldsymbol{e}^3\otimes\boldsymbol{e}^3$. This is because the tensors inside the trace contract over all edges of node i except edge 3, which appear as a tensor product space of $\boldsymbol{e}^3$ with itself due to the outer product nature of the first term and the $\tilde{\mathbf{\Gamma}}$ tensor. Finally, we take the diagonal of the result and add the edge $\boldsymbol{e}^3$ as index to emphasise that it lives in the space spanned by the edge of which we do not compute the expectation with respect to.

Now onto As for the τ parameters, we need to consider the following terms:

$$\mathbb{E}_{q(\Theta/\tau)}[\log p(\boldsymbol{y},\Theta)] = \sum_s^S\mathbb{E}[\log l_s] + \mathbb{E}[\log p(\tau)]$$

$$\mathbb{E}_{q(\Theta/\tau)}[\log p(\boldsymbol{y},\Theta)] = \sum_s^S\mathbb{E}[\log l_s] + \mathbb{E}[\log p(\tau)].$$ (56)

$$\mathbb{E}_{q(\Theta/\tau)}[\log l_s] = \frac{1}{2}\log\tau - \frac{\tau}{2}\mathbb{E}[(y_s - \langle\mathbf{A}\mathbf{X}_s\rangle)^2]=$$ (57)

$$= \frac{1}{2}\log\tau - \frac{\tau}{2}y_s^2 + \tau y_s\mathbb{E}[\langle\mathbf{A}\mathbf{X}_s\rangle] - \frac{\tau}{2}\mathbb{E}[\langle\mathbf{X}_s\mathbf{A}\rangle\langle\mathbf{A}\mathbf{X}_s\rangle]=$$ (58)

$$= \frac{1}{2}\log\tau - \frac{\tau}{2}y_s^2 + \tau y_s\langle\mathbb{E}[\mathbf{A}]\mathbf{X}_s\rangle - \frac{\tau}{2}\langle\mathbb{E}[\mathbf{A}\otimes\mathbf{A}](\mathbf{X}_s\otimes\mathbf{X}_s)\rangle=$$ (59)

$$= \frac{1}{2}\log\tau - \frac{\tau}{2}y_s^2 + \tau y_s\langle\boldsymbol{\mu}\mathbf{X}_s\rangle - \frac{\tau}{2}\langle(\mathbf{P}+\boldsymbol{\mu}\otimes\boldsymbol{\mu}+\boldsymbol{\mu}\otimes\boldsymbol{\mu})(\mathbf{X}_s\otimes\mathbf{X}_s)\rangle=$$ (60)

$$= \frac{1}{2}\log\tau - \frac{\tau}{2}y_s^2 + \tau y_s\langle\boldsymbol{\mu}\mathbf{X}_s\rangle - \frac{\tau}{2}\langle(\mathbf{P}+\boldsymbol{\mu}\otimes\boldsymbol{\mu})(\mathbf{X}_s\otimes\mathbf{X}_s)\rangle+\frac{\tau}{2}\langle(\boldsymbol{\mu}\otimes\boldsymbol{\mu})(\mathbf{X}_s\otimes\mathbf{X}_s)\rangle=$$ (61)

$$= \frac{1}{2}\log\tau - \frac{\tau}{2}([y_s^2 - 2y_s\langle\boldsymbol{\mu}\mathbf{X}_s\rangle + \langle\boldsymbol{\mu}\mathbf{X}_s\rangle^2+\langle(\mathbf{P}+\boldsymbol{\mu}\otimes\boldsymbol{\mu})(\mathbf{X}_s\otimes\mathbf{X}_s)\rangle]=$$ (62)

$$= \frac{1}{2}\log\tau - \frac{\tau}{2}([(y_s - \boldsymbol{\mu}\mathbf{X}_s)^2+\langle(\mathbf{P}+\boldsymbol{\mu}\otimes\boldsymbol{\mu})(\mathbf{X}_s\otimes\mathbf{X}_s)\rangle)\mathbb{E}_{q(\Theta/\tau)}[\log p(\tau)] = (\alpha_\tau^p - 1)\log\tau - \beta_\tau^p\tau],$$ (63)

~~$\sum_{s=1}^{S} \mathbb{E}_{q(\Theta/\tau)}[\log l_s] + \mathbb{E}_{q(\Theta/\tau)}[\log p(\tau)] =$~~
~~$= \sum_{s=1}^{S} \frac{1}{2}\log\tau - \frac{\tau}{2}(\|y_s - \boldsymbol{\mu}\mathbf{X}_s\|^2 - \langle \boldsymbol{\Sigma}(\mathbf{X}_s \otimes \mathbf{X}_s)\rangle) + (\alpha_\tau^p - 1)\log\tau - \beta_\tau^p\tau$~~
~~$= (\frac{S}{2} + \alpha_\tau^p - 1)\log\tau - \tau(\beta_\tau^p + \frac{1}{2}\sum_{s=1}^{S}\|y_s - \boldsymbol{\mu}\mathbf{X}_s\|^2 - \langle \boldsymbol{\Sigma}(\mathbf{X}_s \otimes \mathbf{X}_s)\rangle)$~~

$$\mathbb{E}_{q(\Theta/\tau)}[\log p(\tau)] = (\alpha_\tau^p - 1)\log\tau - \beta_\tau^p\tau. \tag{64}$$

~~And obtaining~~ Summing the previous terms we obtain

$$\sum_{s=1}^{S} \mathbb{E}_{q(\Theta/\tau)}[\log l_s] + \mathbb{E}_{q(\Theta/\tau)}[\log p(\tau)] = \tag{65}$$

$$= \sum_{s=1}^{S} \frac{1}{2}\log\tau - \frac{\tau}{2}(\|y_s - \boldsymbol{\mu}\mathbf{X}_s\|^2 + \langle (\mathbf{P} - \boldsymbol{\mu}\otimes\boldsymbol{\mu})(\mathbf{X}_s \otimes \mathbf{X}_s)\rangle) + (\alpha_\tau^p - 1)\log\tau - \beta_\tau^p\tau = \tag{66}$$

$$= [\frac{S}{2} + \alpha_\tau^p - 1]\log\tau - \tau[\beta_\tau^p + \frac{1}{2}\sum_{s=1}^{S}\|y_s - \boldsymbol{\mu}\mathbf{X}_s\|^2 + \langle (\mathbf{P} - \boldsymbol{\mu}\otimes\boldsymbol{\mu})(\mathbf{X}_s \otimes \mathbf{X}_s)\rangle]. \tag{67}$$

We obtain equation equation 15 ~~using~~ by use of the usual moment matching for gamma distribution.

B Complexity of the node/edge/noise updates.

For the complexity analysis, we consider a reshaping in matrices and vector, this is because otherwise the precise cost of contraction is dependent on the exact shape and size of the tensor. That said we will consider that the contraction will be well optimized, as is fair when using ad-hoc studied python libraries. We can then just use the same computational cost that would happen if the tensors were matricized. Additionally, for the computation we will consider well behaved tensor networks that develop along one dimension, as the one presented in the paper. The reason is that for more complex structures that develop in higher dimensions like Projected Entangled Pair States (PEPS) (Cirac et al., 2021), finding the best contraction is an NP-hard problem and there will be a non reducible computational bottleneck.

Given a generic node $\mathbf{A}^i$ with edge set $E(i)$, each edge dimensions $d_j = \dim(\boldsymbol{e}^j) \quad \forall j \in E(i)$ and total (vectorized) size:

$$R_i = \prod_{j\in E(i)} d_j \tag{68}$$

and define $R = \max(R_i)$ and $d = \max(\dim \boldsymbol{e}^j)$.

The variational parameters attached to the node are the mean $\boldsymbol{\mu}^i \in \bigotimes_{j\in E(i)} V_j$ and the second moment $\mathbf{P}^i \in \bigotimes_{j\in E(i)}(V_j \otimes V_j)$, whose vectorization and matricization are of size R_i and $R_i \times R_i$ respectively, so storing a single node costs $\sim \mathcal{O}(R_i^2)$ memory.

The node update equation 13 requires:

- calculating and accumulating the data statistic $\sum_s \hat{\mathbf{P}}^i \mathbf{X}_s^P$ of the environment, assuming that when optimized we can consider all nodes contractions as having the same cost, dominated by the contraction of the sigma network. We can upper bound the cost of contracting a node to the environment to be given by $R_i \dim(\boldsymbol{e}^j)$ where $\boldsymbol{e}^j$ is the contracting edge. The complexity of each node addition to the $\boldsymbol{\mu}$ network is upper bounded by Rd, which is repeated for $N-1$ nodes and

S samples, obtaining $\mathcal{O}(S(N-1)Rd)$. For the $\mathbf{P}$ network the edges are contracted in couples, so both the node size and the contracting bond are squared, giving R^2d^2 per node addition and $\mathcal{O}(S(N-1)R^2d^2)$ overall. So the total cost is $\mathcal{O}(S(N-1)(R^2d^2+Rd))$, dominated by the $\mathbf{P}$ network term $\mathcal{O}(S(N-1)R^2d^2)$.

- adding the diagonal edge-precision term $\tilde{\mathbf{\Gamma}}^{E(i)}$, $\mathcal{O}(R_i^2)$
- inverting the resulting $R_i \times R_i$ precision to obtain $\mathbf{\Sigma}^{i\star}$, $\mathcal{O}(R_i^3)$
- the mean update $\boldsymbol{\mu}^{i\star} = \mathbb{E}[\tau]\mathbf{\Sigma}^{i\star}\sum_s y_s \hat{\boldsymbol{\mu}}^i \mathbf{X}_s^\mu$ is composed by a matrix-vector product $\mathcal{O}(R_i^2)$ plus an $\mathcal{O}(S\,(N-1)Rd)$ accumulation of the right-hand side.

The node update costs is therefore approximately upper bounded by:

$$\mathcal{O}\big(S\,(N-1)(R^2d^2+Rd)+R^3\big) \quad \text{time}, \qquad \mathcal{O}\big(R^2\big) \quad \text{memory}, \tag{69}$$

where the inversion of the single node dominates whenever $R \gtrsim S(N-1)d^2$, while for large sample sizes the $\mathbf{P}$ network accumulation $S(N-1)R^2d^2$ is the leading term. While this is true we should also consider that the network accumulation cost due to the number of nodes N, with properly designed algorithm is reduced by storing intermediate passages during computation and reusing partial contractions.

The edge update equation 14 for edge j needs the partial trace $tr_{E(i)/j}[\mathbf{P}^i\tilde{\mathbf{\Gamma}}^{E(i)/j}]_{\boldsymbol{e}^i,\boldsymbol{e}^i}$, which contracts the second moment $\mathbf{P}^i$ against the diagonal precisions of the *other* edges of $\mathbf{A}^i$, and approximately contributes as $\mathcal{O}(R_i^3)$, while the concentration/rate $(\boldsymbol{\alpha}_j, \boldsymbol{\beta}_j)$ update is $\mathcal{O}(d_j)$.

Summed over the edges of the node,

$$\sum_{\boldsymbol{e}^j \in E(i)} \mathcal{O}(R_i^3) = \mathcal{O}(|E(i)|\,R_i^3). \tag{70}$$

The noise update equation 15 contributes the residual $\sum_s \|y_s - \langle \boldsymbol{\mu}\mathbf{X}_s^\mu\rangle\|^2$ and the epistemic term $\sum_s \langle(\mathbf{P}-\boldsymbol{\mu}\otimes\boldsymbol{\mu})\mathbf{X}_s^P\rangle$, where the contribution is dominated by the $\mathbf{P}$ network accumulation.

Hence the upper bound of the cost for a full sweep over the updates of one node is dominated by

$$\mathcal{O}\big(S\,(N-1)(R^2d^2+Rd)+R^3\big), \tag{71}$$

the node mean and covariance update.

For complex network, since the cost of the contraction path is tied to the defined structure and dominated by the contraction of the second moment network, we can generalize by defining $C_{\mathbf{P}}$ as the contraction cost of the network $\mathbf{P}$ for one sample and obtain

$$\mathcal{O}\big(SC_{\mathbf{P}}+R^3\big). \tag{72}$$

C Uncertainty Quantification

A point prediction tells us where a model expects the target to be, but it says nothing about how much that expectation should be trusted. For Bayesian models we care equally about the predicted value and about the associated confidence. In this section we evaluate the quality of the predictive uncertainty produced by the BTNR method and compare it against the Bayesian baselines GP, BDE, HSBNN and BASS. We first define the metrics that quantify uncertainty quality, retrieved from Bayesian deep learning literature (Lakshminarayanan et al., 2017; Kuleshov et al., 2018; Tran et al., 2020), and then present the figures and tables that report the described metrics.

C.1 Predictive distribution and notation

Every model is assumed to output, for each test point $s \in \{1, \dots, S\}$, a Gaussian predictive distribution

$$y_s \sim \mathcal{N}\big(\mu_s,\ \sigma_s^2\big), \tag{73}$$

where S is the number of test points, μ_s is the predictive mean and σ_s the predictive standard deviation. We write Φ and ϕ for the standard normal cumulative distribution and density functions, and $F_s(z) = \Phi\big((z - \mu_s)/\sigma_s\big)$ for the predictive cumulative distribution function at point s.

We evaluate ten regression tasks. Each combination of model and dataset is repeated over $K = 10$ random seeds.

For a scalar metric m with per seed values $\{m^{(1)}, \dots, m^{(K)}\}$ we report the empirical mean and the sample standard deviation across seeds,

$$\bar{m} = \frac{1}{K} \sum_{k=1}^{K} m^{(k)}, \qquad \mathrm{std}(m) = \sqrt{\frac{1}{K-1} \sum_{k=1}^{K} \big(m^{(k)} - \bar{m}\big)^2}. \tag{74}$$

The BTNR entry in every figure and table is the best BTNR configuration for that dataset ~~and metric~~reaching the best pointwise estimate on the validation set (highest R^2).

C.2 Uncertainty metrics

Probabilistic accuracy. The negative log-likelihood (NLL) is the most widely reported scoring rule for regression uncertainty (Gal and Ghahramani, 2016; Lakshminarayanan et al., 2017). It measures how probable the observed targets are under the predictive distribution,

$$\mathrm{NLL} = \frac{1}{S} \sum_{s=1}^{S} \frac{1}{2} \Big(\log\big(2\pi\sigma_s^2\big) + \frac{(y_s - \mu_s)^2}{\sigma_s^2}\Big), \tag{75}$$

with lower values indicating a better fit. The continuous ranked probability score (CRPS) is a scoring rule introduced by Matheson and Winkler (1976) and widely adopted following the analysis of Gneiting and Raftery (2007). It measures the distance between the predictive cumulative distribution function and the empirical one,

$$\mathrm{CRPS} = \frac{1}{S} \sum_{s=1}^{S} \int_{-\infty}^{\infty} \big(F_s(z) - \mathbf{1}[\, z \geq y_s \,]\big)^2 \, \mathrm{d}z. \tag{76}$$

For the Gaussian predictive distribution the per point integral has the closed form (Gneiting and Raftery, 2007)

$$\mathrm{CRPS}_s = \sigma_s \Big[\frac{y_s - \mu_s}{\sigma_s} \Big(2\,\Phi\big(\tfrac{y_s - \mu_s}{\sigma_s}\big) - 1\Big) + 2\,\phi\big(\tfrac{y_s - \mu_s}{\sigma_s}\big) - \frac{1}{\sqrt{\pi}}\Big]. \tag{77}$$

It is a proper scoring rule expressed in the units of y, more robust to outliers than NLL, and again lower is better. The results related to probabilistic accuracy are reported in Tables 2 and 3.

Calibration. Calibration tries to quantify whether the predicted uncertainty has the correct size (Guo et al., 2017; Kuleshov et al., 2018). The expected calibration error (ECE) was first proposed for classification through reliability estimates (Pakdaman Naeini et al., 2015; Guo et al., 2017) and later extended to regression by checking the coverage of predicted quantiles (Kuleshov et al., 2018). We fix a grid of quantile levels $P = \{0.1, 0.2, \dots, 1.0\}$, where each level $p \in P$ is a nominal probability. For a given p the model is well calibrated if a fraction p of the targets falls below its predicted p-quantile. We define ECE in the main paper in 6 Equation 18. A value near zero means the model is well calibrated.

The prediction interval coverage probability (PICP) measures how often the targets fall inside the nominal 95% interval (Pearce et al., 2018),

$$\mathrm{PICP} = \frac{1}{S}\sum_{s=1}^{S} \mathbf{1}[\,\mu_s - z\,\sigma_s \le y_s \le \mu_s + z\,\sigma_s\,], \qquad z = \Phi^{-1}(0.975) \approx 1.96, \tag{78}$$

and is best when it equals its target of 0.95. The sharpness is the mean predicted standard deviation,

$$\mathrm{SH} = \frac{1}{S}\sum_{s=1}^{S} \sigma_s, \tag{79}$$

which captures how confident the model is: a smaller value means tighter predictive distributions.

Both PICP and sharpness are only meaningful once calibration is satisfied, since a model can be arbitrarily sharp while being badly miscalibrated. Results for calibration are shown in Figure 7, Figure 8 and Table 4.

Discrimination. Discrimination evaluates whether the uncertainty is effectively associating predictions by their error. We order the test points by predicted standard deviation in descending order and, for a removal fraction $r \in [0, 1]$, compute the root mean squared error on the points that remain,

$$\mathrm{RMSE}_{\mathrm{unc}}(r) = \sqrt{\frac{1}{|R_r|}\sum_{s\in R_r}(y_s - \mu_s)^2}, \qquad R_r = \{\text{points kept after removing the } \lceil rS \rceil \text{ most uncertain}\}. \tag{80}$$

The oracle curve $\mathrm{RMSE}_{\mathrm{oracle}}(r)$ performs the same operation but removes points in order of their true absolute error, giving the best achievable ordering. Normalising each curve by its value at $r = 0$, $\widehat{\mathrm{RMSE}}(r) = \mathrm{RMSE}(r)/\mathrm{RMSE}(0)$, the area under the sparsification error (AUSE) is the area between the uncertainty and oracle curves (Ilg et al., 2018),

$$\mathrm{AUSE} = \int_0^1 \left(\widehat{\mathrm{RMSE}}_{\mathrm{unc}}(r) - \widehat{\mathrm{RMSE}}_{\mathrm{oracle}}(r)\right) \mathrm{d}r, \tag{81}$$

where lower is better. As a complementary, measure the does not depend on thresholds, we also report the Spearman correlation ρ_{S} between the predicted standard deviations $\{\sigma_s\}$ and the absolute errors $\{|y_s - \mu_s|\}$. To compute it, each of the two quantities is sorted in ascending order and be indicated according to their position, from 1 for the smallest to S for the largest, ρ_{S} is then the ordinary Pearson correlation between these two position sequences rather than between the raw values. Results for discrimination are shown in Figure 6 and Table 5.

Outlier detection. These metrics are evaluated by inject synthetic outliers into the test set and probe whether the uncertainty can flag them as outliers, using the predictive standard deviation σ_s or the per point NLL as an anomaly score a_s (Hendrycks and Gimpel, 2017).

Treating the injected points as the positive class $y_s \in \{0, 1\}$, a threshold τ on the score induces a true positive rate, false positive rate and precision,

$$\mathrm{TPR}(\tau) = \frac{\sum_s y_s\,\mathbf{1}[a_s \ge \tau]}{\sum_s y_s}, \quad \mathrm{FPR}(\tau) = \frac{\sum_s (1-y_s)\,\mathbf{1}[a_s \ge \tau]}{\sum_s (1-y_s)}, \quad \mathrm{Prec}(\tau) = \frac{\sum_s y_s\,\mathbf{1}[a_s \ge \tau]}{\sum_s \mathbf{1}[a_s \ge \tau]}. \tag{82}$$

We report the area under the resulting ROC curve (AUROC, the area under TPR against FPR) and the area under the precision vs recall curve (AUPR, the area under Prec against TPR), both of which increase as detection improves. An uninformative score gives AUROC = 0.5. Outlier detection results are reported in Table 6.

Uncertainty decomposition. For BTNR we additionally separate the predictive standard deviation into an epistemic part σ_{epi}, which reflects reducible uncertainty due to limited data or model capacity, and an

Table 2: Test negative log-likelihood (NLL) (Equation 75) (lower is better), mean ± std. Best per dataset in bold. The table shows that BTNR attains competitive likelihoods across all ten tasks and remains stable even where some baselines diverge to large values

Model	AB	AI	AP	BK	CO	EE	OB	RS	SB	SP
BTNR	2.15 ± 0.01	-0.65 ± 0.00	5.75 ± 0.01	6.05 ± 0.00	3.35 ± 0.08	0.82 ± 0.04	1.41 ± 0.04	3.19 ± 0.03	9.65 ± 0.41	2.48 ± 0.00
GP	2.43 ± 0.01	-0.67 ± 0.00	359.08 ± 7.04	244.53 ± 3.37	7.54 ± 0.00	2.24 ± 0.00	1.25 ± 0.01	9.04 ± 0.00	22967.24 ± 330.04	3.00 ± 0.00
BDE	**1.97** ± 0.01	**-2.51** ± 0.48	**5.03** ± 0.02	**4.55** ± 0.00	**2.74** ± 0.03	**0.44** ± 0.06	**-0.92** ± 0.22	**3.10** ± 0.03	**6.36** ± 0.03	2.62 ± 0.05
HSBNN	2.31 ± 0.01	-0.59 ± 0.00	5.98 ± 0.00	6.22 ± 0.02	3.87 ± 0.01	3.31 ± 0.01	1.73 ± 0.01	3.68 ± 0.01	7.55 ± 0.01	**2.45** ± 0.01
BASS	111.04 ± 28.15	81.98 ± 37.75	143.13 ± 23.30	96.12 ± 19.17	24.22 ± 4.54	19.05 ± 1.81	36.76 ± 21.93	13.37 ± 2.18	88.53 ± 16.50	85.71 ± 26.22

aleatoric part σ_{ale}, which reflects irreducible noise in the data (Kendall and Gal, 2017; Gal and Ghahramani, 2016). The scale free epistemic fraction summarises how much of the uncertainty is reducible,

$$\rho_{\text{epi}} = \frac{\sigma_{\text{epi}}}{\sigma_{\text{epi}} + \sigma_{\text{ale}}}, \tag{83}$$

where a large value signals uncertainty that could be reduced with more data or capacity and a small value signals inherent label noise. Uncertainty decomposition is shown for BTNR in Figure 9.

C.3 Uncertainty experiments

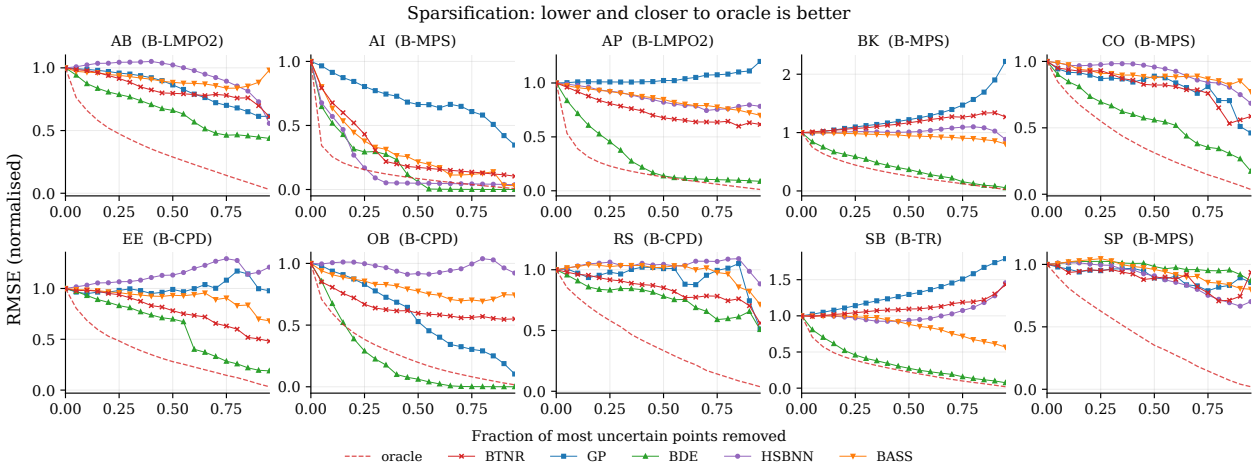


Figure 6: The plots address whether the uncertainty ranks points by error (Ilg et al., 2018). For each model we draw the normalised RMSE by uncertainty curve $\widehat{\text{RMSE}}_{\text{unc}}(r)$ against the removal fraction r, together with the oracle curve $\widehat{\text{RMSE}}_{\text{oracle}}(r)$, the area between them is the AUSE (see Equation 81). A good model produces an uncertainty curve that drops quickly and stays close to the oracle~~, and the~~. The BTNR entry ~~in each panel~~ is the configuration with highest R^2 on the ~~lowest AUSE for that dataset~~validation set.

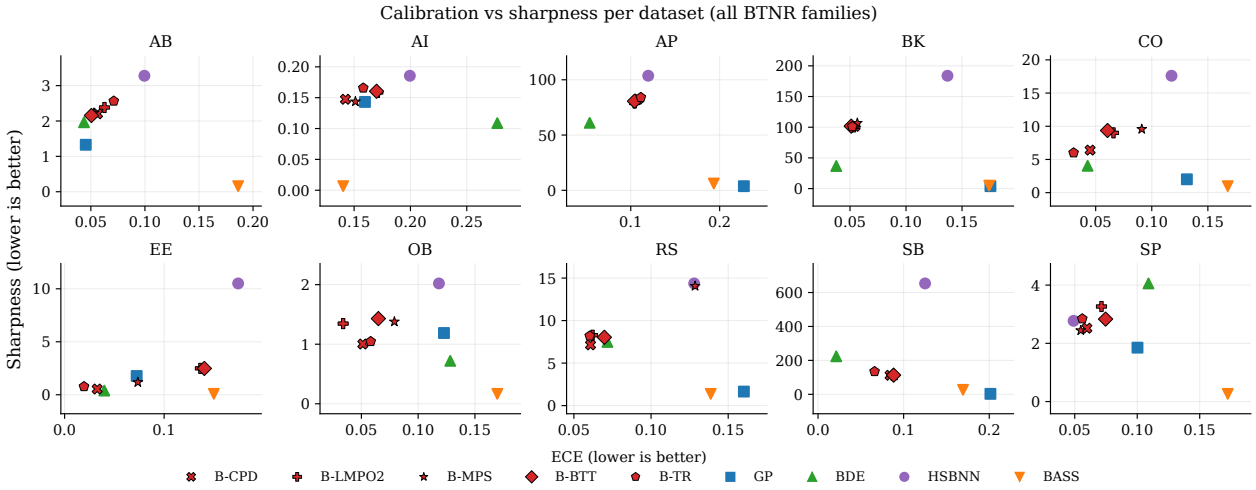


Figure 7: Each marker places a model at $x = \text{ECE}$ (see Equation 18) and $y = \text{SH}$ (see Equation 79), so the bottom left corner corresponds to the desirable combination of low calibration error and high confidence. The plot shows the raw pairs without normalisation, one panel per dataset. A model should be both calibrated and sharp, hence the bottom left corner (the origin) is the desirable combination of low calibration error and high confidence.~~For each dataset we report which tensor network structures is plotted.~~

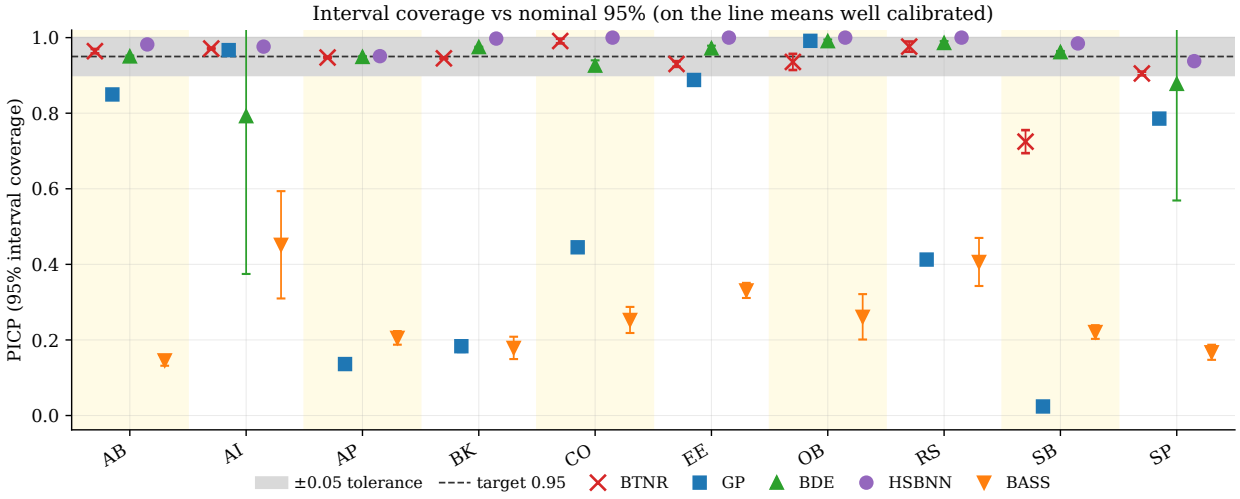


Figure 8: This figure checks whether the nominal 95% intervals contain 95% of the targets. For each model and dataset we plot the PICP (see Equation 78), with markers at the seed mean and error bars at the seed standard deviation. A dashed line marks the target of 0.95 inside a shaded tolerance band of ±0.05. Points on the line are well calibrated, points below indicate overconfidence and points above indicate underconfidence. The BTNR entry is the configuration ~~whose PICP is closest to 0.95 for each dataset~~with highest R^2 on the validation set. The dashed line marks the target of 0.95 inside a shaded tolerance band of ±0.05.

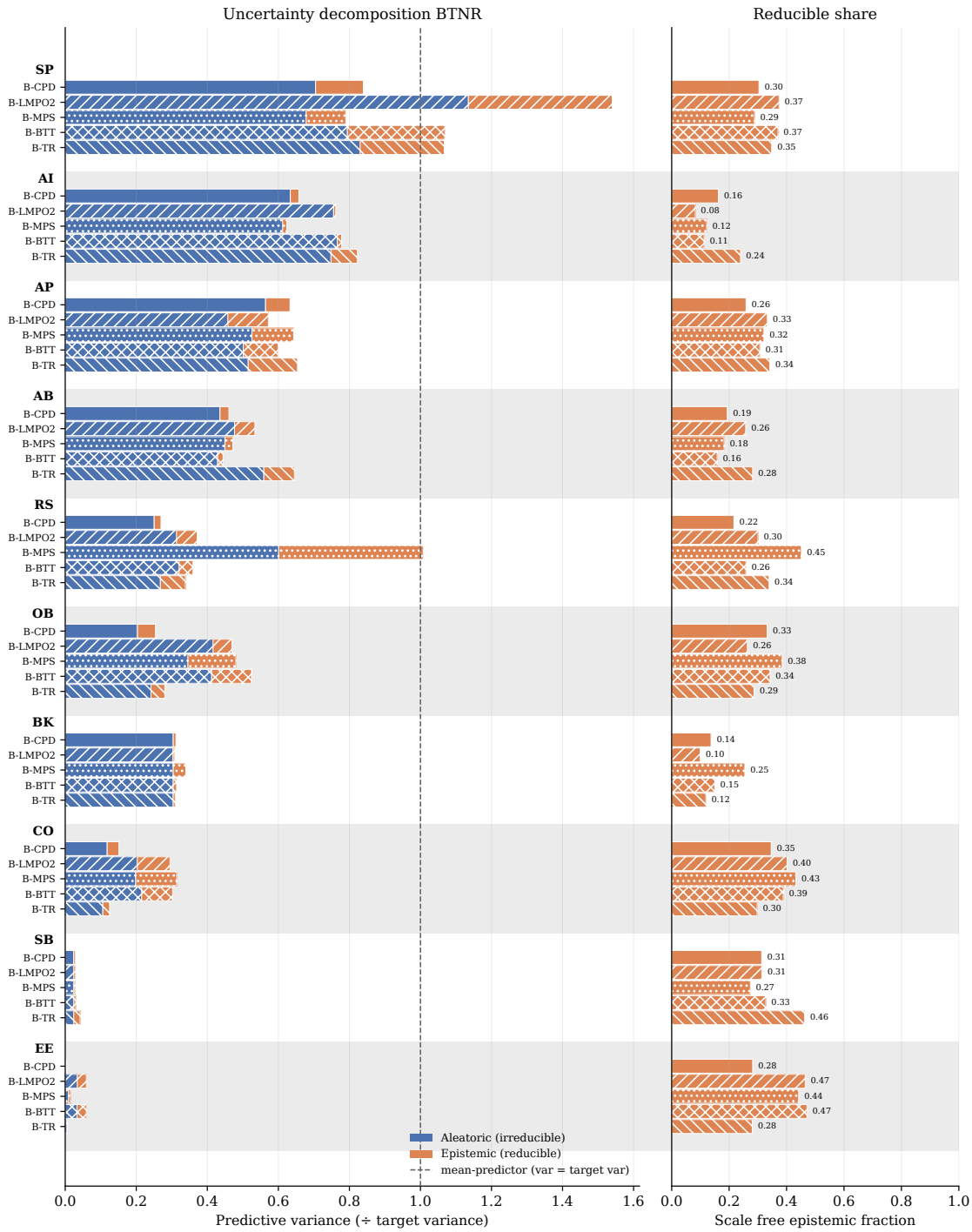


Figure 9: Uncertainty decomposition for BTNR. This figure, shows how much of the predictive uncertainty is reducible. ~~Using the NLL best BTNR structure per dataset, the~~ The left panel stacks the mean aleatoric and epistemic ~~standard deviations σ_{ale}~~ variance σ^2_{ale} and ~~σ_{epi}~~ σ^2_{epi} in target units, with datasets sorted by their total. The right panel shows the scale free epistemic fraction ρ_{epi} (~~see~~ Equation 83)~~defined above~~.

Table 3: Test continuous ranked probability score (CRPS) (Equation 76) (lower is better), mean ± std. Confirm the comparable results of BTNR in Table 2.

Model	AB	AI	AP	BK	CO	EE	OB	RS	SB	SP
BTNR	1.11 ± 0.01	0.06 ± 0.00	37.10 ± 0.21	53.90 ± 0.07	3.58 ± 0.21	0.30 ± 0.01	0.56 ± 0.03	3.22 ± 0.12	181.94 ± 1.86	1.62 ± 0.00
GP	1.11 ± 0.00	0.05 ± 0.00	51.84 ± 0.61	47.19 ± 1.19	4.65 ± 0.00	1.17 ± 0.00	0.42 ± 0.00	4.27 ± 0.00	351.50 ± 1.68	1.90 ± 0.00
BDE	**1.03** ± 0.00	**0.04** ± 0.00	**32.12** ± 0.15	**17.90** ± 0.13	**2.37** ± 0.06	**0.21** ± 0.01	**0.27** ± 0.02	**2.98** ± 0.05	**116.64** ± 0.77	1.82 ± 0.07
HSBNN	1.21 ± 0.01	0.06 ± 0.00	45.98 ± 0.14	54.62 ± 1.98	5.30 ± 0.04	2.74 ± 0.03	0.64 ± 0.02	4.37 ± 0.05	221.10 ± 2.91	**1.59** ± 0.01
BASS	1.41 ± 0.01	**0.04** ± 0.00	50.69 ± 0.33	39.05 ± 1.42	4.24 ± 0.11	0.33 ± 0.00	0.80 ± 0.05	3.90 ± 0.14	189.85 ± 3.37	2.22 ± 0.03

Table 4: Expected calibration error (ECE) (Equation 18) (lower is better). Best per dataset in bold. The table highlights the calibration behaviour: BTNR reaches the lowest calibration error on several datasets and stays reasonably close to the best elsewhere, which indicates that its predictive intervals have a reliable size.

Model	AB	AI	AP	BK	CO	EE	OB	RS	SB	SP
BTNR	0.063 ± 0.005	0.151 ± 0.006	0.104 ± 0.001	0.057 ± 0.001	0.091 ± 0.011	**0.033** ± 0.007	**0.051** ± 0.006	**0.061** ± 0.007	0.066 ± 0.010	0.055 ± 0.001
GP	0.045 ± 0.008	0.160 ± 0.006	0.227 ± 0.010	0.175 ± 0.004	0.131 ± 0.000	0.072 ± 0.000	0.123 ± 0.003	0.160 ± 0.000	0.201 ± 0.001	0.100 ± 0.000
BDE	**0.044** ± 0.001	0.277 ± 0.144	**0.054** ± 0.001	**0.038** ± 0.002	**0.043** ± 0.004	0.040 ± 0.005	0.128 ± 0.005	0.072 ± 0.005	**0.021** ± 0.002	0.109 ± 0.155
HSBNN	0.100 ± 0.002	0.199 ± 0.004	0.120 ± 0.002	0.137 ± 0.008	0.118 ± 0.002	0.174 ± 0.004	0.118 ± 0.007	0.128 ± 0.004	0.125 ± 0.005	**0.049** ± 0.005
BASS	0.186 ± 0.003	**0.140** ± 0.045	0.193 ± 0.003	0.174 ± 0.004	0.168 ± 0.009	0.150 ± 0.010	0.170 ± 0.014	0.139 ± 0.014	0.169 ± 0.002	0.172 ± 0.008

Table 5: Discrimination metrics. The area under the sparsification error (AUSE) (Equation 81) (lower is better) and the Spearman correlation between predicted uncertainty and absolute error (higher is better). Best per dataset within each block is in bold.

Model	AB	AI	AP	BK	CO	EE	OB	RS	SB	SP
AUSE										
BTNR	0.955 ± 0.063	0.021 ± 0.003	41.927 ± 0.557	80.769 ± 0.610	2.483 ± 0.109	0.229 ± 0.023	0.404 ± 0.030	2.429 ± 0.165	271.222 ± 2.801	1.256 ± 0.017
GP	0.988 ± 0.036	0.068 ± 0.004	77.639 ± 2.801	69.969 ± 1.046	2.694 ± 0.000	1.390 ± 0.000	0.206 ± 0.009	3.338 ± 0.000	508.673 ± 17.499	1.476 ± 0.000
BDE	**0.603** ± 0.011	0.025 ± 0.003	**15.255** ± 0.334	**7.170** ± 0.086	**1.327** ± 0.045	**0.121** ± 0.011	**0.069** ± 0.019	**1.909** ± 0.130	**39.516** ± 1.967	1.473 ± 0.168
HSBNN	1.212 ± 0.035	**0.011** ± 0.002	57.785 ± 2.843	48.563 ± 8.182	3.252 ± 0.160	2.147 ± 0.391	0.532 ± 0.067	3.999 ± 0.441	252.236 ± 10.867	**1.237** ± 0.065
BASS	1.117 ± 0.146	0.023 ± 0.009	56.245 ± 2.425	33.984 ± 1.401	2.838 ± 0.362	0.278 ± 0.053	0.521 ± 0.100	3.686 ± 0.431	172.940 ± 16.596	1.444 ± 0.108
Spearman										
BTNR	0.171 ± 0.023	0.546 ± 0.056	0.348 ± 0.009	-0.213 ± 0.009	0.222 ± 0.041	0.269 ± 0.125	0.407 ± 0.038	0.107 ± 0.066	-0.067 ± 0.014	**0.182** ± 0.011
GP	0.165 ± 0.016	0.188 ± 0.054	-0.051 ± 0.043	-0.481 ± 0.056	0.274 ± 0.000	0.079 ± 0.000	0.405 ± 0.020	0.019 ± 0.000	-0.530 ± 0.033	0.087 ± 0.000
BDE	**0.443** ± 0.010	**0.938** ± 0.040	**0.653** ± 0.005	**0.662** ± 0.006	**0.439** ± 0.028	**0.594** ± 0.050	**0.911** ± 0.015	**0.226** ± 0.050	**0.697** ± 0.009	0.122 ± 0.321
HSBNN	-0.020 ± 0.020	0.593 ± 0.065	0.185 ± 0.028	0.021 ± 0.090	0.150 ± 0.042	-0.122 ± 0.137	0.040 ± 0.105	-0.108 ± 0.079	0.145 ± 0.032	0.078 ± 0.041
BASS	0.090 ± 0.058	0.714 ± 0.070	0.196 ± 0.025	0.074 ± 0.039	0.075 ± 0.086	0.103 ± 0.151	0.285 ± 0.103	-0.031 ± 0.047	0.187 ± 0.070	-0.069 ± 0.102

Table 6: Outlier detection averaged across the ten datasets. The table reports the outlier detection metrics AUROC and AUPR (Equation 82), under both anomaly scores, averaged across the ten datasets. Because the per dataset values are highly uniform the error bars are small. Models are rows and the four metrics are columns, and each cell is the mean over datasets of the per dataset seed mean plus or minus the standard deviation across datasets. The results show that detection is governed almost entirely by the choice of anomaly score rather than by the model: the per point NLL, which contains the residual term $(y_s - \mu_s)^2/\sigma_s^2$, flags the corrupted targets almost perfectly for every method (AUROC and AUPR near one), whereas the predictive standard deviation alone, which does not see the corrupted target, stays close to chance for all methods.

Model	AUROC (std)	AUPR (std)	AUROC (NLL)	AUPR (NLL)
BTNR	**0.504** ± 0.024	**0.125** ± 0.026	**0.999** ± 0.002	**0.985** ± 0.037
GP	0.488 ± 0.050	0.108 ± 0.015	**0.999** ± 0.002	0.977 ± 0.046
BDE	0.494 ± 0.031	0.115 ± 0.012	0.984 ± 0.035	0.966 ± 0.064
HSBNN	0.494 ± 0.016	0.120 ± 0.021	**0.999** ± 0.002	0.980 ± 0.042
BASS	0.503 ± 0.019	0.121 ± 0.020	0.998 ± 0.003	0.983 ± 0.027

D Expanded Experimental Section

Note that in the code provided with the paper (Anonymous Repository, 2026), all configurations used to reproduce the results are available and used by the code under 'conf/' directory.

D.1 Hyperparameter ablation

All methods are evaluated across:

- **Seeds**: $\{42, 7, 123, 256, 999\}$ (5 ~~random~~ seeds)
- **Datasets**: concrete, abalone, energy_efficiency, realstate, student_perf, obesity, bike, appliances, ai4i, seoulBike (10 datasets)
- $L \in \{3, 4\}$ (polynomial degree of feature map)

Tensor Networks Methods

~~MPO2~~MPS, LMPO2, BTT, TR with ALS training:

- ~~$L \in \{3, 4\}$ (polynomial degree of feature map)~~
- $r \in \{6, 12, 18\}$ (edge dimension)
- ~~$\alpha \in \{1.0, 5.0\}$~~ $\alpha \in \{0.01, 0.1, 1.0, 10, 100\}$ (edge prior strength, regularization for ALS)
- ~~$\sigma_{\text{init}} \in \{0.01, 0.1\}$ (initialization strength)~~ $\sigma_{\text{init}} \in \{1.0, 0.01\}$

ALS-CPD (CPD with ALS training)

- ~~$L \in \{3, 4\}$ (polynomial degree of feature map)~~
- $r \in \{18, 32, 64\}$ (edge dimension)
- ~~$\alpha \in \{1.0, 5.0\}$~~ $\alpha \in \{0.01, 0.1, 1.0, 10, 100\}$ (edge prior strength, regularization for ALS)
- ~~$\sigma_{\text{init}} \in \{0.01, 0.1\}$ (initialization strength)~~ $\sigma_{\text{init}} \in \{1.0, 0.01\}$

~~MPO2~~MPS, LMPO2, BTT, TR with BTNR training

- ~~$L \in \{3, 4\}$ (polynomial degree of feature map)~~
- $r = 18$ (edge dimension)
- ~~$\alpha \in \{1.0, 5.0\}$~~ $(\alpha, \sigma_{\text{init}}) \in \{(5.0, 0.1), (1.0, 0.01)\}$ (edge prior strength~~)~~
- ~~$\sigma_{\text{init}} \in \{0.01, 0.1\}$ (~~, initialization strength)

CPD with BTNR training

- ~~$L \in \{3, 4\}$ (polynomial degree of feature map)~~
- $r = 64$ (edge dimension)
- ~~$\alpha \in \{1.0, 5.0\}$~~ $(\alpha, \sigma_{\text{init}}) \in \{(5.0, 0.1), (1.0, 0.01)\}$ (edge prior strength~~)~~, initialization strength)

D.2 Hyperparameter test

Each model specification is chosen per dataset, selecting the structure reaching mean highest validation quality.

The test seeds are:

- ~~$\sigma_{\text{init}} \in \{0.01, 0.1\}$ (initialization strength~~**Seeds**: {923, 2143, 2415, 3094, 3176, 6758, 7994, 9086, 9153, 9958} (10 seeds)

Baseline Methods

We compare against four established probabilistic regression baselines, each providing a predictive mean together with a calibrated predictive standard deviation.

Gaussian Process (GP) (MacKay et al., 1998). A non parametric Bayesian model that places a prior directly over functions through a covariance (kernel) function. The posterior predictive distribution is available in closed form, and the kernel hyperparameters are fitted by maximizing the marginal log likelihood. We use a squared exponential (RBF) or Matérn 5/2 kernel, optionally with automatic relevance determination (per dimension lengthscales). On larger datasets, where exact inference scales cubically in the number of points, we instead employ a sparse variational GP with inducing points. We utilize the gpytorch implementation (Gardner et al., 2018).

Bayesian Deep Ensemble (BDE) (Sommer et al., 2025). We adopt the Microcanonical Langevin Ensembles (MILE) approach, which performs sampling based inference over the weights of a Bayesian neural network. Each ensemble member is initialized by a deterministic optimization warm up and then explored with Microcanonical Langevin Monte Carlo (MCLMC), which bypasses the Metropolis Hastings accept/reject step for faster and more predictable traversal of the multimodal posterior. Uncertainty is obtained by aggregating the posterior predictive samples across the chains. We utilize the implementation in (bde).

Horseshoe Bayesian Neural Network (HSBNN) (Overweg et al., 2019). A single hidden layer Bayesian neural network equipped with a tied, sparsity inducing Horseshoe prior on the input weights, yielding automatic relevance determination and interpretable feature selection. The posterior is approximated by variational inference (Bayes by Backprop layers together with the Horseshoe scale hyperparameters), and the predictive distribution is recovered through Monte Carlo sampling. We utilize the implementation in (Microsoft)

Bayesian Adaptive Smoothing Splines (BASS) (Friedman, 1991). A non parametric Bayesian regression model that represents the response as an adaptive expansion in tensor product spline basis functions, in the spirit of (Bayesian) multivariate adaptive regression splines. The number of basis functions, their interaction order, and their knot locations are inferred jointly through reversible jump Markov chain Monte Carlo, so that model complexity is selected automatically from the data. We use the public pyBASS implementation (Los Alamos National Laboratory, 2021).

~~BDE~~ **BDE** (Bayesian Deep Ensemble)

- Hidden layers $\in \{[16, 16], [32, 32]\}$
- Number of ensemble members $M \in \{4, 8\}$

~~Exact GP~~ **Exact GP** (Gaussian Process)

- Kernel type $\in \{\text{RBF}, \text{Matérn}\}$
- ARD $\in \{\text{true}, \text{false}\}$

Note: Only evaluated on smaller datasets (concrete, energy_efficiency, realstate, student_perf).

~~Sparse GP~~ **Sparse GP** (Sparse Gaussian Process)

- Number of inducing points $n_{\text{ind}} \in \{50, 100\}$
- Kernel type $\in \{\text{RBF}, \text{Matérn}\}$

Note: Only evaluated on larger datasets (abalone, obesity, bike, appliances, ai4i, seoulBike).

~~Horseshoe BNN~~ **Horseshoe BNN** (Bayesian Neural Network)

- Number of hidden units $n_h \in \{32, 64\}$
- Number of posterior samples $n_s \in \{5, 10\}$

~~BASS~~ **BASS**

- Maximum interaction order $d_{\max} \in \{2, 3\}$
- Maximum number of basis functions $B_{\max} \in \{30, 50\}$

In Table 7 we report the shorhand notation of the datasets and their size. All data is normalized.

UCI ID	Code	Dataset	Train	Val	Test	Feat.
374	AP	Appliances Energy Prediction	13814	2960	2961	27
275	BK	Bike Sharing	12165	2607	2607	12
601	AI	AI4I	7000	1500	1500	9
560	SB	Seoul Bike Sharing	6132	1314	1314	18
1	AB	Abalone	2923	627	627	11
544	OB	Obesity Levels	1477	317	317	39
165	CO	Concrete Compressive Strength	721	154	155	8
242	EE	Energy Efficiency	537	115	116	8
320	SP	Student Performance	454	97	98	50
477	RS	Real Estate Valuation	289	62	63	6

Table 7: Datasets with shorthand codes used, tasks, number of observations, and number of features.

In Figure 10 we use a one dimensional function to visualize the predictions and uncertainty quantification for each sample point.

In Table 8 we plot the time of an ablation study in a toy scenario, where an ALS model is ablated over all configuration of edge dimensions in a finite set {1,2,3} and then report the best result in the validation set, compared with a run of BTNR. This result represents an extreme case of the model selection cost, since a full ablation over all possible combinations of bond dimensions is rarely performed in practice. More commonly, a single bond dimension is chosen and shared across the network, and the model is evaluated for different values of this common bond dimension. While this substantially reduces the search cost, it constrains the space of tensor network architectures that can be explored, potentially limiting performance and preventing the identification of the best trade off between predictive performance and model compression.

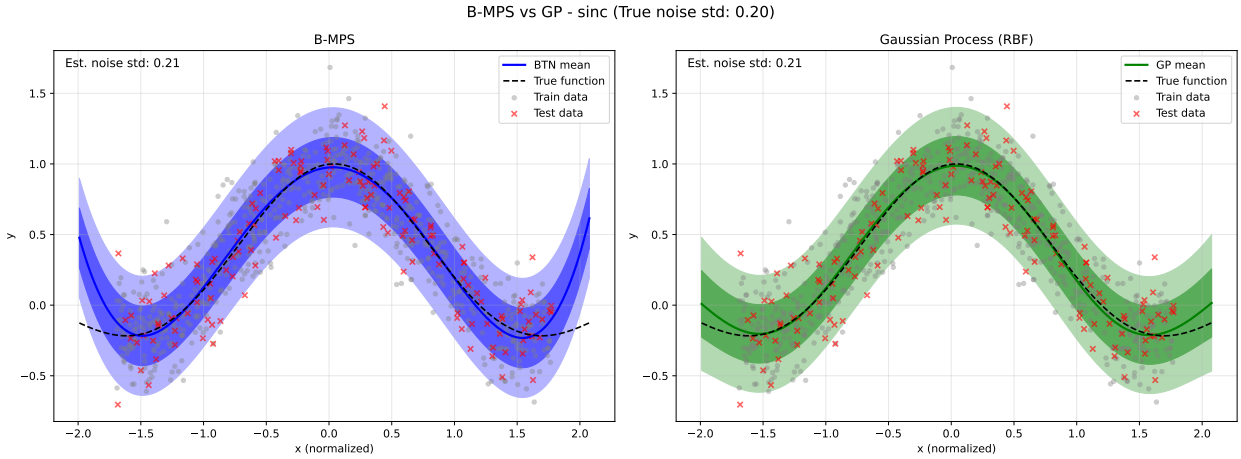


Figure 10: Comparison of prediction and riconstruction error of synthetic data generated using $\sin(\pi x)/\pi x$ and artificially added noise with std 0.2.

	ALS	BTNR
~~sec~~ height Time (in seconds)	2781	9
quality (R^2)	0.83	0.82

Table 8: LMPO2, third degree polynomial feature map, ablation edge dimension set {1,2,3}.